

# Lecture 8

PARALLELISM BASICS

CS336

Tatsu H

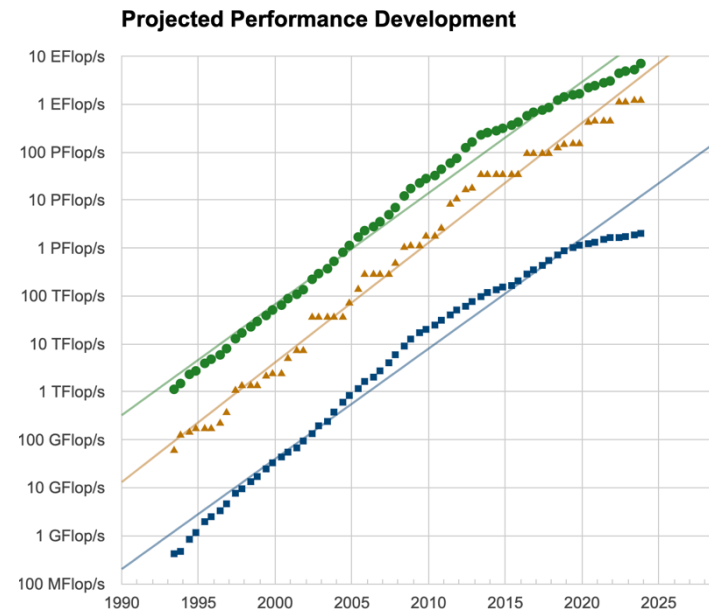
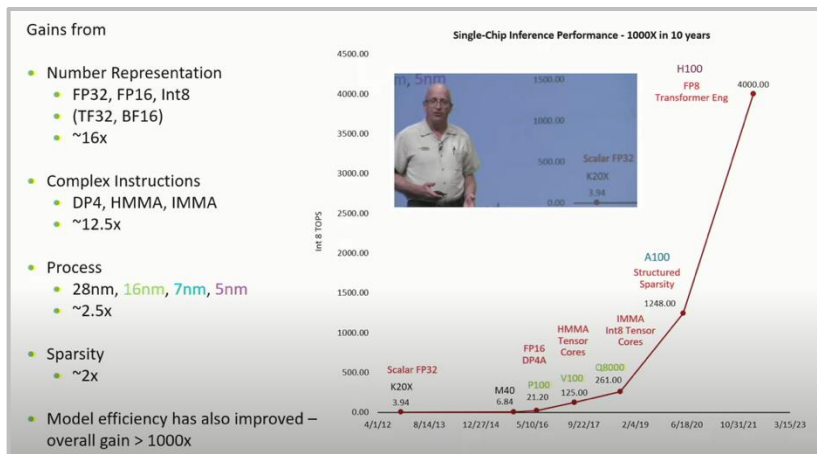
## Outline and goals

- Understand the systems complexities of training huge models
- Different parallelization paradigms and why people use multiple at once
- What large scale training runs often look like

# Organization today:

- ❖ **Part 1:** Basics of networking for LLMs
- ❖ **Part 2:** Different forms of parallel LLM training
- ❖ **Part 3:** Scaling and training big LMs with parallelism

# Limits to GPU-based scaling – compute

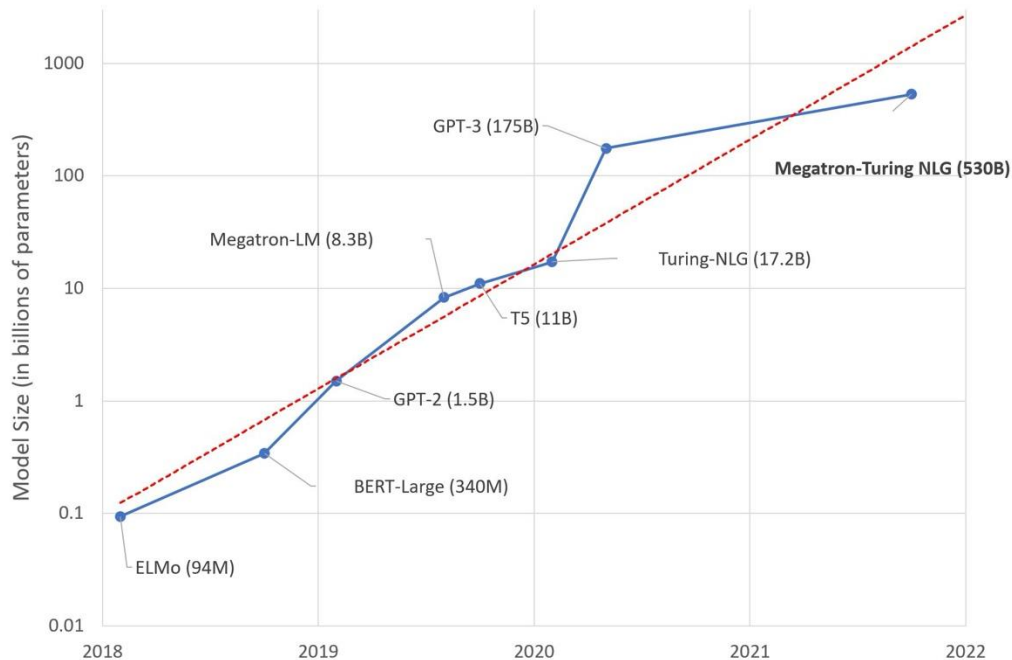


There are limits to single-GPU scaling.

The world's fastest supercomputers have *exaflops* of compute

# Limits to GPU-based scaling - memory

Models are getting really big..

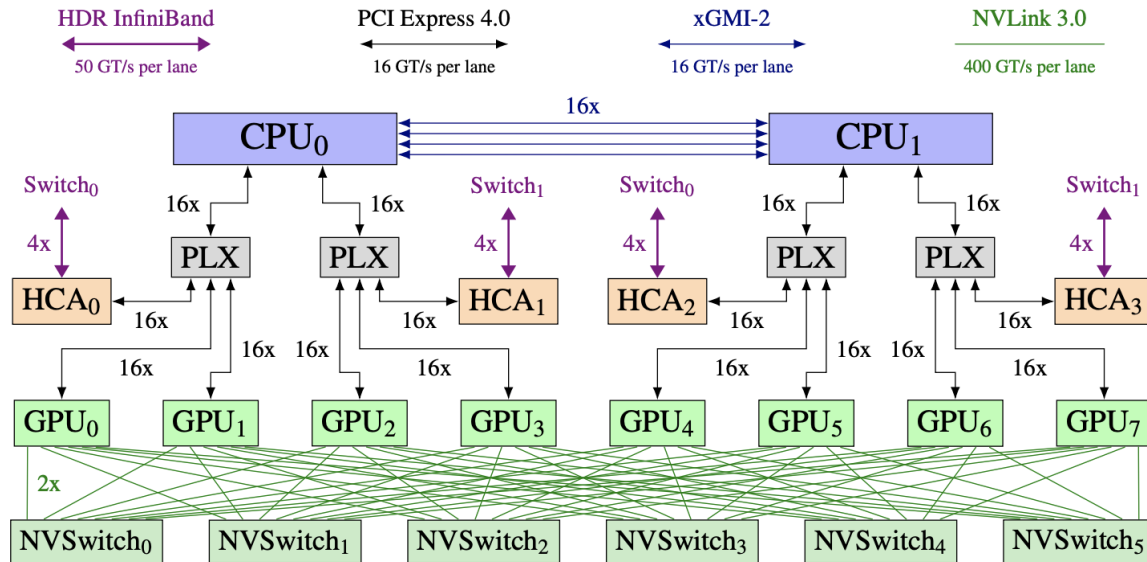


A single GPU can't fit most of these large models!

# What do we do? Multi-GPU, multi-machine parallelism

Intra-node parallelism  
via high-speed interconnects

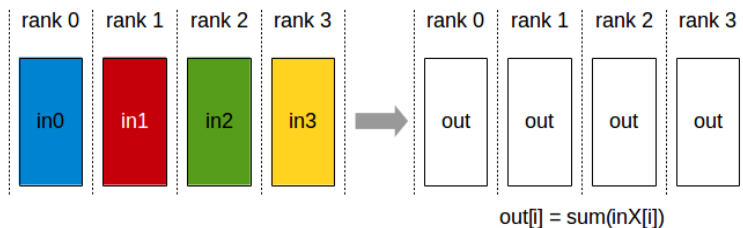
High-speed inter-node parallelism



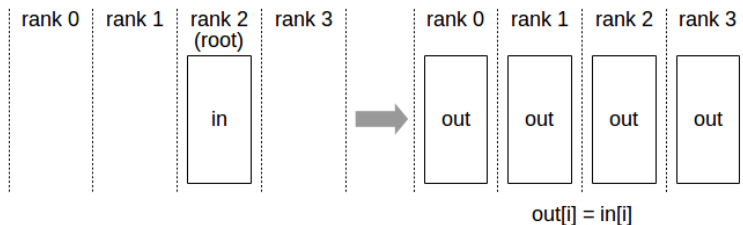
Split up memory and compute requirements across GPUs and machines

# But first.. Some basics about collective communication

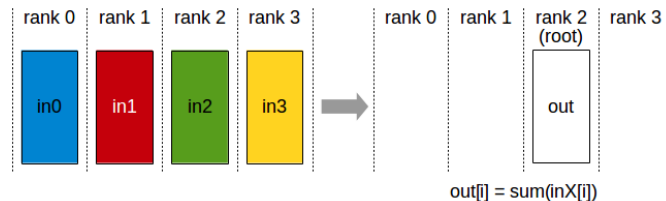
## All reduce



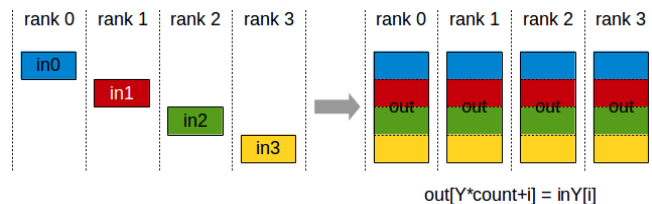
## Broadcast



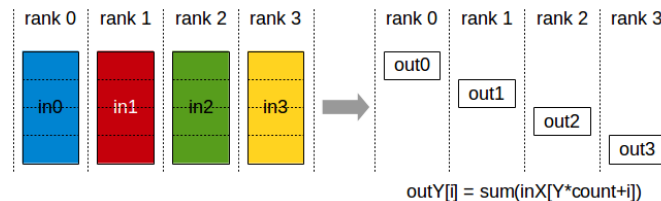
## Reduce



## All Gather

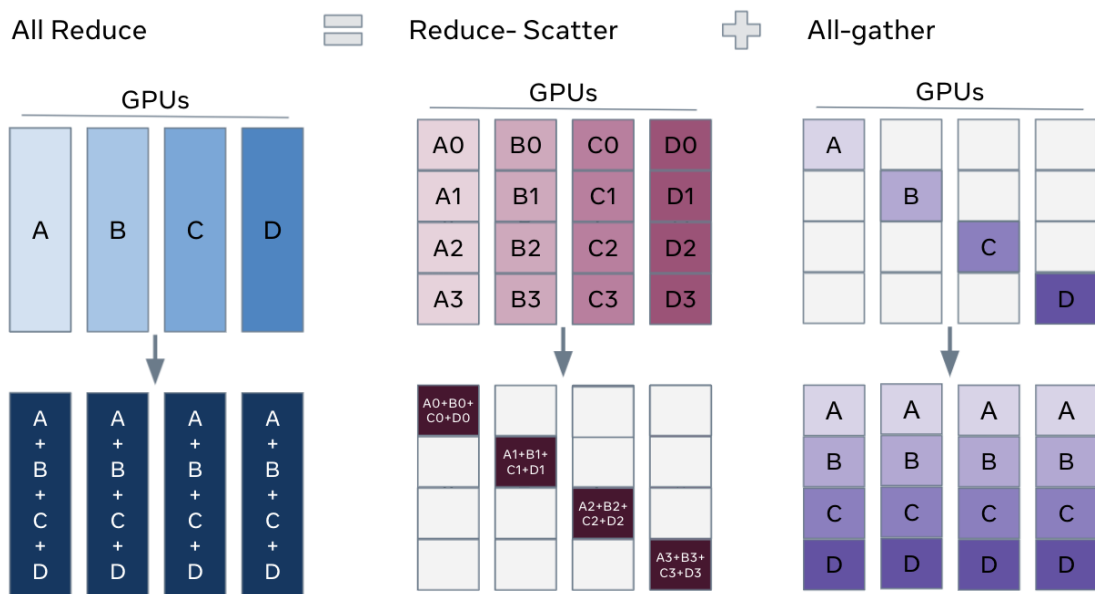


## Reduce Scatter



# Important detail – all reduce vs reduce-scatter-gather.

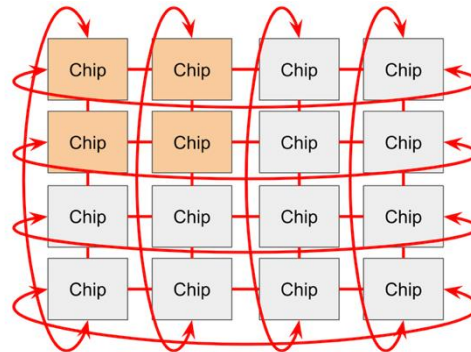
Reduce can be implemented as two steps: reduce-scatter and all-gather



Importantly, in the bandwidth-limited regime, this is the best you can do

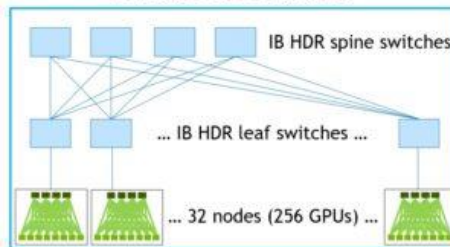
# TPUs vs GPUs – design differences at the comm level

**TPU networking**  
toroidal mesh

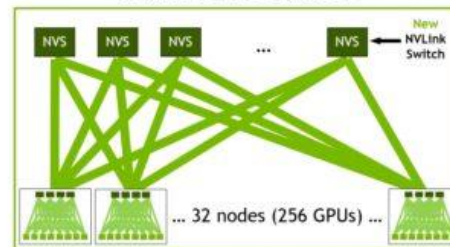


**GPU networking**  
All-to-all (up to 256)

DGX A100 256 SuperPOD



DGX H100 256 SuperPOD



Fully NVLink-connected  
Massive bisection bandwidth

|                    | A100 SuperPod    |                     |                  | H100 SuperPod    |                     |                  | Speedup   |        |
|--------------------|------------------|---------------------|------------------|------------------|---------------------|------------------|-----------|--------|
|                    | Dense<br>PFLOP/s | Bisection<br>[GB/s] | Reduce<br>[GB/s] | Dense<br>PFLOP/s | Bisection<br>[GB/s] | Reduce<br>[GB/s] | Bisection | Reduce |
| 1 DGX / 8 GPUs     | 2.5              | 2,400               | 150              | 16               | 3,600               | 450              | 1.5x      | 3x     |
| 32 DGXs / 256 GPUs | 80               | 6,400               | 100              | 512              | 57,600              | 450              | 9x        | 4.5x   |

# Mesh vs tree vs other

## Why mesh? Why tree?

Pro mesh – Cheaper to operate, can be made fast (and just do tensor parallel)

Pro tree/A2A – Better for less structured communications (expert parallel)

taekim.substack.com/p/nvidias-bill-dally-and-googles-jeff

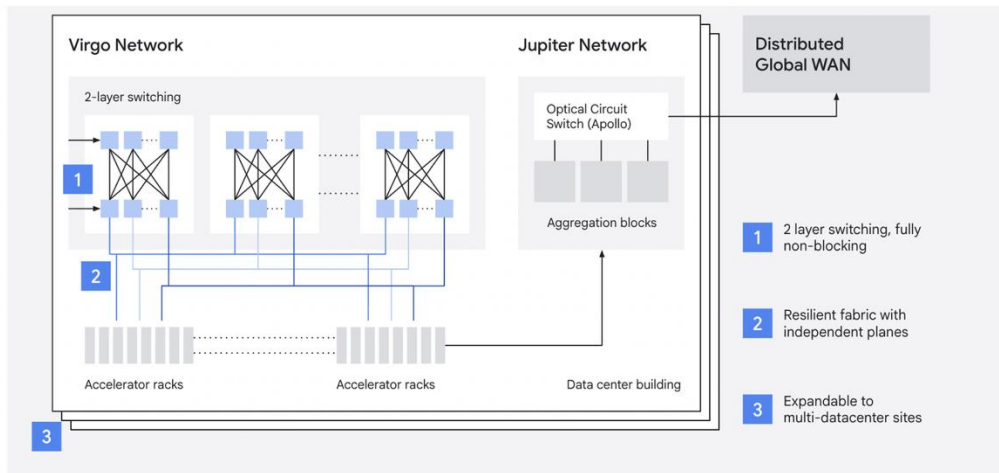
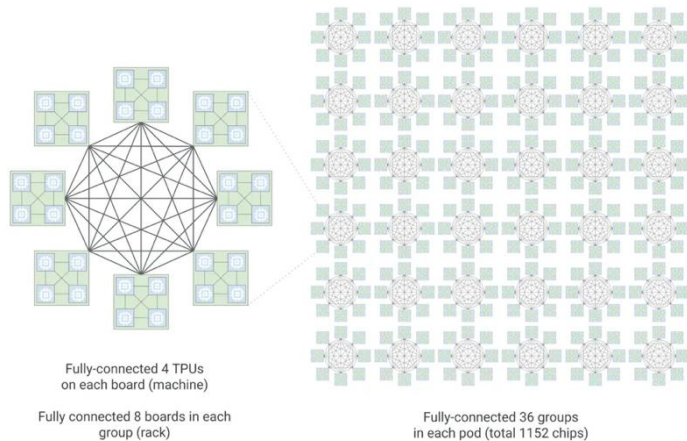
Dally and Dean on Google TPUs versus Nvidia GPUs. GPUs are good for ..

Mixture of Experts models and TPUs are good for local workloads

Dally: “You can’t say one is better than the other because it really has to do with what is the workload and what is the traffic pattern. If you have a very local workload, a direct connected network with relatively low radix like a 3D torus, is ideal because you can map your problem. So part of the problem is on this TPU and part of the problem is on this TPU and you have to go one hop to get your data over there. But then there are other cases where, for example, you have an MOE [mixture of experts] model and you’ve got a lot of experts, and they’re scattered all over the place and now you’re having to take many hops to get to a given expert. So it would be more efficient to do one hop up to a switch and one hop down to the expert that you want to be at” “Given a traffic pattern, you’d come up with an optimum network, but there’s no network that is good for all traffic patterns.”

Dean: “I totally agree. It depends on exactly what your workload is.”

## But then things change.. TPU8i/t



**TPU8i networking** – closer to tree-style topologies (maybe for MoEs?)

**TPU8t scale-out networking** – ‘Virgo’ with switched networks

# Why not connect everything?

## Domain sizes – why not connect everything?

| Key Capabilities - Huawei Ascend 910C Cloud Matrix 384 vs Nvidia GB200 NVL72 |             |                    |                    |                  |
|------------------------------------------------------------------------------|-------------|--------------------|--------------------|------------------|
| Chip and Package Level                                                       |             |                    |                    |                  |
|                                                                              | Unit        | GB200              | Ascend 910C        | Huawei vs Nvidia |
| BF16 dense TFLOPS                                                            | TFLOPS      | 2,500              | 780                | 0.3x             |
| HBM capacity                                                                 | GB          | 192                | 128                | 0.7x             |
| HBM bandwidth                                                                | TB/s        | 8.0                | 3.2                | 0.4x             |
| Scale Up Bandwidth                                                           | Gb/s uni-di | 7,200              | 2,800              | 0.4x             |
| Scale Out Bandwidth                                                          | Gb/s uni-di | 400                | 400                | 1.0x             |
| System Level                                                                 |             |                    |                    |                  |
|                                                                              | Unit        | Nvidia GB200 NVL72 | Cloud Matrix CM384 | Huawei vs Nvidia |
| BF16 dense PFLOPS                                                            | PFLOPS      | 180                | 300                | 1.7x             |
| HBM capacity                                                                 | TB          | 13.8               | 49.2               | 3.6x             |
| HBM bandwidth                                                                | TB/s        | 576                | 1,229              | 2.1x             |
| Scale Up Bandwidth                                                           | Gb/s uni-di | 518,400            | 1,075,200          | 2.1x             |
| Scale Up Domain Size                                                         | GPUs        | 72                 | 384                | 5.3x             |
| Scale Out Bandwidth                                                          | Gb/s uni-di | 28,800             | 153,600            | 5.3x             |
| All-In System Power <sub>1</sub>                                             | W           | 145,000            | 599,821            | 4.1x             |
| All-in Power per BF16 dense FLOP                                             | W/TFLOP     | 0.81               | 2.00               | 2.5x             |
| All-in Power per memory bandwidth                                            | W per TB/s  | 251.7              | 488.1              | 1.9x             |
| All-in Power per memory capacity                                             | kW/TB       | 10.5               | 12.2               | 1.2x             |

1. All-in System Power is total cluster power including scale-out networking, storage, etc.



## Part 1 recap

- ❖ New unit of compute – the datacenter
- ❖ What we want from multi-machine scaling:
  - ❖ Linear memory scaling (max model params scales with num gpus)
  - ❖ Linear compute scaling (model flops scale linearly with num gpus)
- ❖ Simple collective comms primitives

## Part 2 – Standard LLM parallelization primitives

How do we parallelize LLMs? 3 important ideas

- **Data parallelism**
  - Naïve data parallel
  - ZeRO levels 1-3
- **Model parallelism**
  - Pipeline parallel
  - Tensor parallel
- **Activation parallelism**
  - Sequence parallel

# Naïve data parallelism

**Starting point** – imagine we are doing naïve SGD

$$\theta_{t+1} = \theta_t - \eta \sum_{i=1}^B \nabla f(x_i)$$

**Naïve parallelism:** split the elements of B sized batch across M machines. Exchange gradients to synchronize

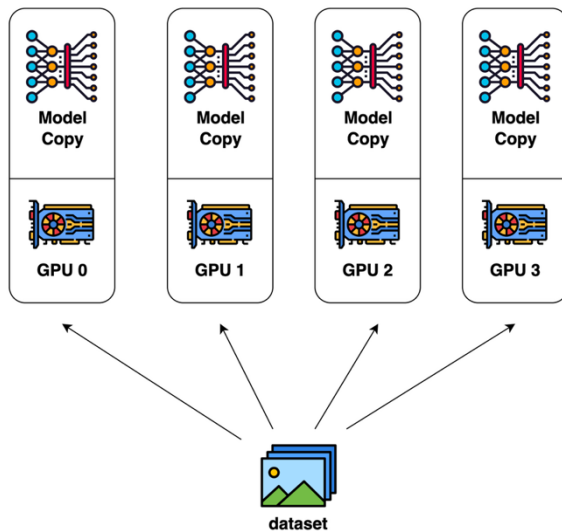
## How does this do?

Compute scaling – each GPU gets B/M examples.

Communication overhead – transmits 2x # params every batch. OK if batches are big

Memory scaling – none. Every GPU needs # params at least

# What's wrong with naïve data parallel?



Memory seems like it'd be a problem – we copy the model parameters to each GPU.  
Let's take a closer look..

# What's wrong with naïve data parallelism? - Memory

**Our memory situation is actually *terrible*.**

Depending on our precision..

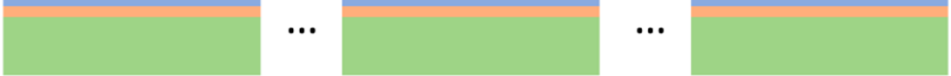
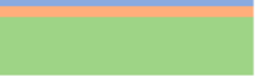
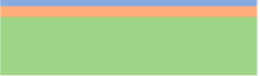
We need 5 copies of weights and 16 bytes per param!

- 2 bytes for FP/BF 16 model parameters
- 2 bytes for FP/BF 16 gradients
- 4 bytes for FP32 master weights (the thing you accumulate into in SGD)
- 4 (or 2) bytes for FP32/BF16 Adam first moment estimates
- 4 (or 2) bytes for FP32/BF16 Adam second moment estimates

“Optimizer state”

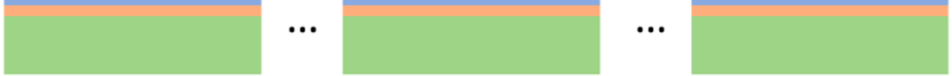
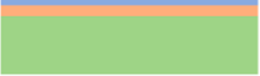
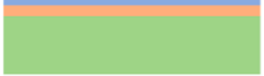
# ZeRO – solving the memory overhead issue of DP

**Core idea:** split up the expensive parts (state) and use the reduce-scatter equivalence.

|                     | gpu <sub>0</sub>                                                                   | ... | gpu <sub>i</sub>                                                                   | ... | gpu <sub>N-1</sub>                                                                  | Memory Consumed                        | K=12<br>Ψ=7.5B<br>N <sub>d</sub> =64 |
|---------------------|------------------------------------------------------------------------------------|-----|------------------------------------------------------------------------------------|-----|-------------------------------------------------------------------------------------|----------------------------------------|--------------------------------------|
| Baseline            |  | ... |  | ... |  | $(2 + 2 + K) * \Psi$                   | 120GB                                |
| P <sub>os</sub>     |  | ... |  | ... |  | $2\Psi + 2\Psi + \frac{K * \Psi}{N_d}$ | 31.4GB                               |
| P <sub>os+g</sub>   |  | ... |  | ... |  | $2\Psi + \frac{(2 + K) * \Psi}{N_d}$   | 16.6GB                               |
| P <sub>os+g+p</sub> |  | ... |  | ... |  | $\frac{(2 + 2 + K) * \Psi}{N_d}$       | 1.9GB                                |

■ Parameters   
 ■ Gradients   
 ■ Optimizer States

## ZeRO stage 1. optimizer state sharding

|                 | gpu <sub>0</sub>                                                                   | ... | gpu <sub>i</sub>                                                                  | ... | gpu <sub>N-1</sub>                                                                  | Memory Consumed                        | K=12<br>$\Psi=7.5\text{B}$<br>$N_d=64$ |
|-----------------|------------------------------------------------------------------------------------|-----|-----------------------------------------------------------------------------------|-----|-------------------------------------------------------------------------------------|----------------------------------------|----------------------------------------|
| Baseline        |  | ... |  | ... |  | $(2 + 2 + K) * \Psi$                   | 120GB                                  |
| P <sub>os</sub> |  | ... |  | ... |  | $2\Psi + 2\Psi + \frac{K * \Psi}{N_d}$ | 31.4GB                                 |

### High level idea:

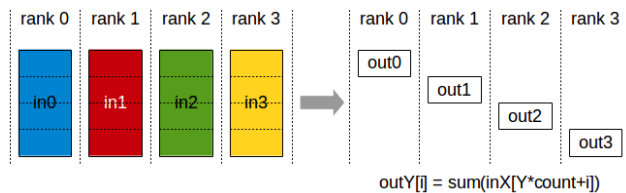
- Split up the optimizer state (first + second moments) across GPUs
- Everyone has the parameters + gradients

Each worker is responsible for updating a subset of params (corresponding to their slice)

# ZeRO stage 1. how it works

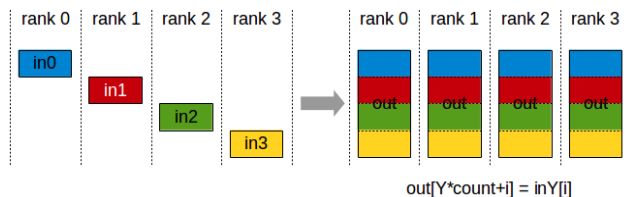
**Step 1.** Everyone computes a full gradient on their subset of the batch

**Step 2.** ReduceScatter the gradients – incur #params communication cost



**Step 3.** Each machine updates their param using their gradient + state.

**Step 4.** All Gather the parameters – incur #params communication cost



## Comparing ZeRO stage 1 and naïve data parallel

|                         | Naïve DDP                   | ZeRO stage 1                                                      |
|-------------------------|-----------------------------|-------------------------------------------------------------------|
| Communication primitive | One all-reduce (gradients)  | One reduce scatter (send gradients) + all gather (collect params) |
| Communication cost      | $2 * \# \text{ params}$     | $2 * \# \text{ params}$                                           |
| Memory                  | $(4+K) * \# \text{ params}$ | $(4+K/N_{\text{gpu}}) * \# \text{ params}$                        |

ZeRO stage 1 is *free* (in the bandwidth limited regime) memory wins

## ZeRO stage 2. the simple extension to gradient sharding



Emboldened by our success, let's shard even more stuff

### High level idea

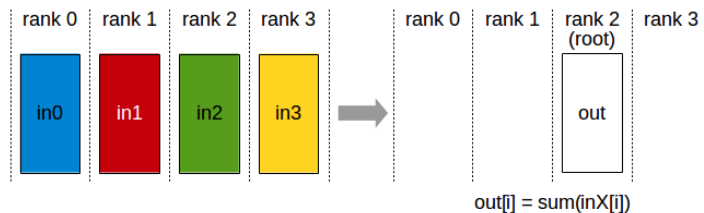
- Also keep the gradients (pink slices) sharded across the machines.
- Use the same (rough) tricks as stage 1.

**Complexity** – we can never instantiate a full gradient vector, but each worker must compute a full gradient (since we're data parallel)

## ZeRO stage 2. how it works

**Step 1.** Everyone incrementally goes backward on the computation graph

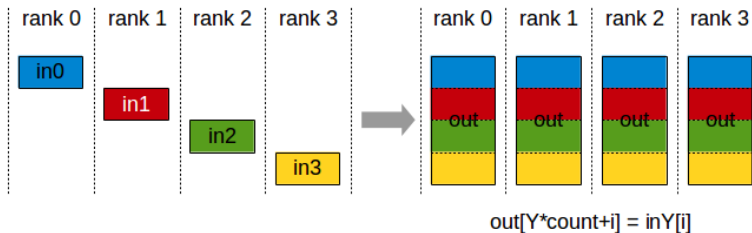
Step 1a. After computing a layer's gradients, immediately reduce to send this to the right worker



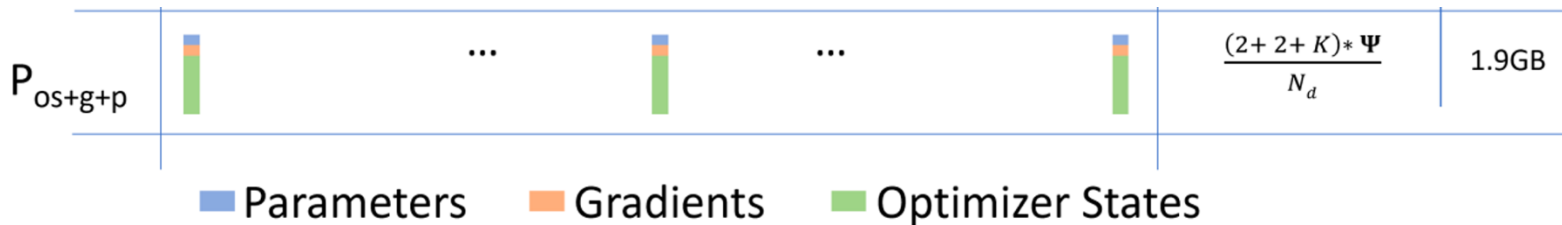
Step 1b. Once gradients are not needed in the backward graph, immediately free it.

**Step 2.** Each machine updates their param using their gradient + state.

**Step 3.** All Gather the parameters.



## ZeRO stage 3 (aka FSDP) shard *everything*



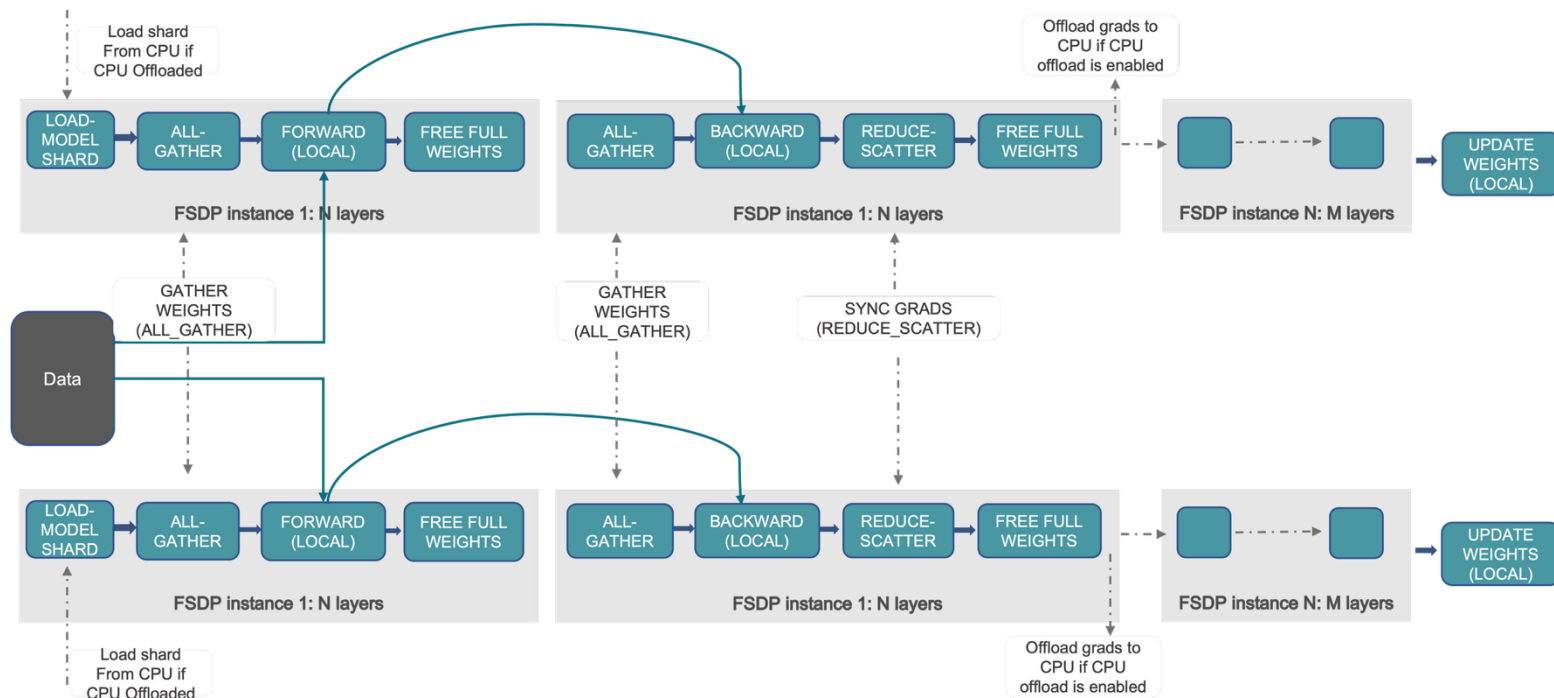
We've gotten almost everything for free so far.. lets try to solve *all* our memory issues

### High level idea

- Shard everything – incl parameters!
- Use the same 'incremental communication / computation' ideas
- Send and request parameters on demand while stepping through the compute graph.

Is it possible to do this with low overhead?

## ZeRO stage 3 (aka FSDP) how it works (baby version)



Communication cost – 2 all gather (#param), 1 reduce-scatter (#param).

# Actual picture of how FDSP / ZeRO stage 3 works

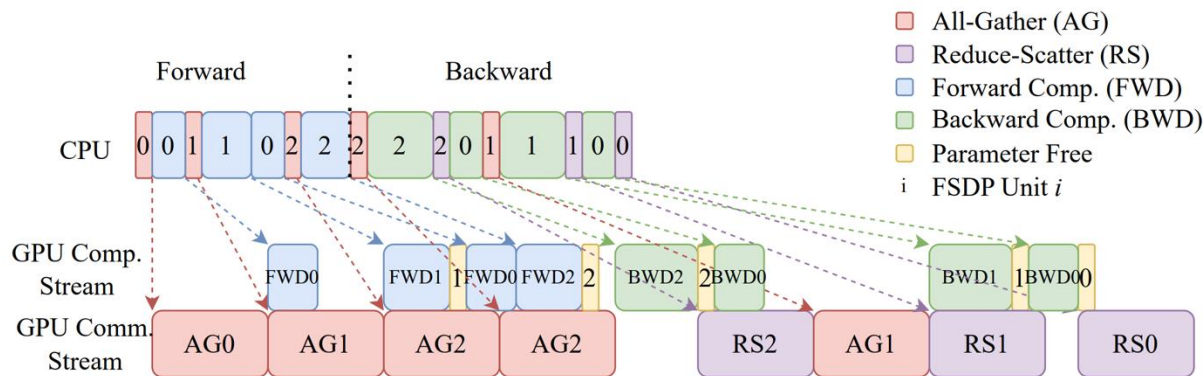
Let's walk through a FSDP example to see some important ideas

## Incremental computation / communication

- Parameters / gradients are requested / sent and then immediately freed

## Overlapping communication and computation ( $W_1W_0 + W_2W_0$ ) $x = y$

- The all-gathers happen all at once while forward happens, masking the comm cost.



## What's the point?

Distributed data parallel costs  $2 \times \#$  param communication

### What about ZeRO?

- **Zero stage 1** is  $2 \times \#$  param – it's free! – you might as well always do it
- **Zero stage 2** is  $2 \times \#$  param – it's (almost) free (ignoring overhead)
- **Zero stage 3** is  $3 \times \#$  param – 1.5x comm cost, but that's not bad! (ignoring latency..)

This is also conceptually very simple – write a FSDP block wrapper.

## ZeRO in practice: will it fit?

Pure BF16 training (with Kahan summation), is viable and optimizer states are less beefy. Let's say BF16 for everything but the master weights – 12 bytes per param

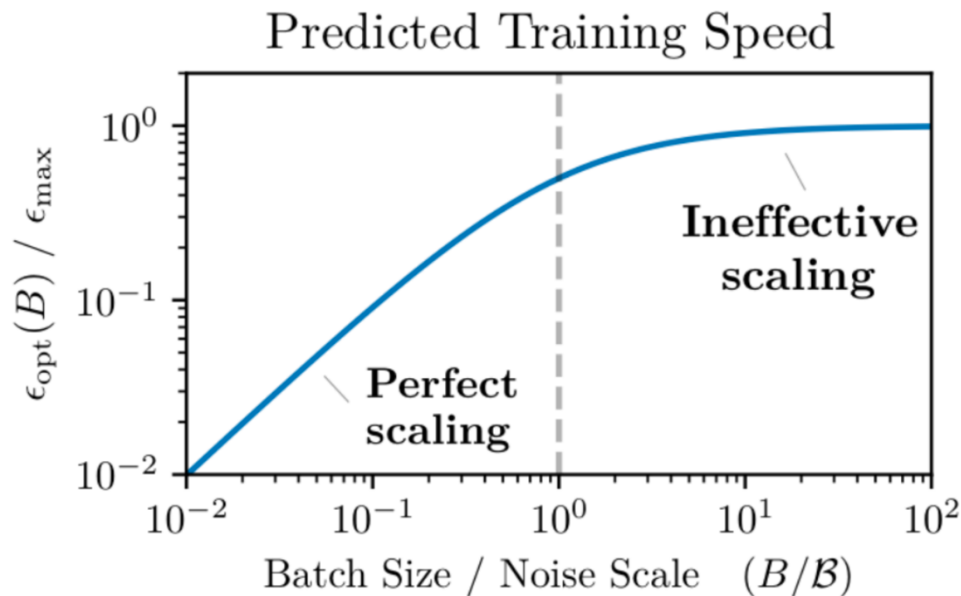
On a 8X A100 80G..

|              | Max size (params) | Formula for B/param                               |
|--------------|-------------------|---------------------------------------------------|
| Baseline     | 6.66.. B          | 12                                                |
| Zero stage 1 | 16 B              | 5                                                 |
| Zero stage 2 | 24.62 B           | $2 \text{ (param)} + (10 \text{ (grad+state)})/8$ |
| Zero stage 3 | 53.33 B           | 12/8                                              |

## Issues remain with data parallel – compute scaling

With data parallel, **#machines** < **batch size** (and near this, comm overhead is high)

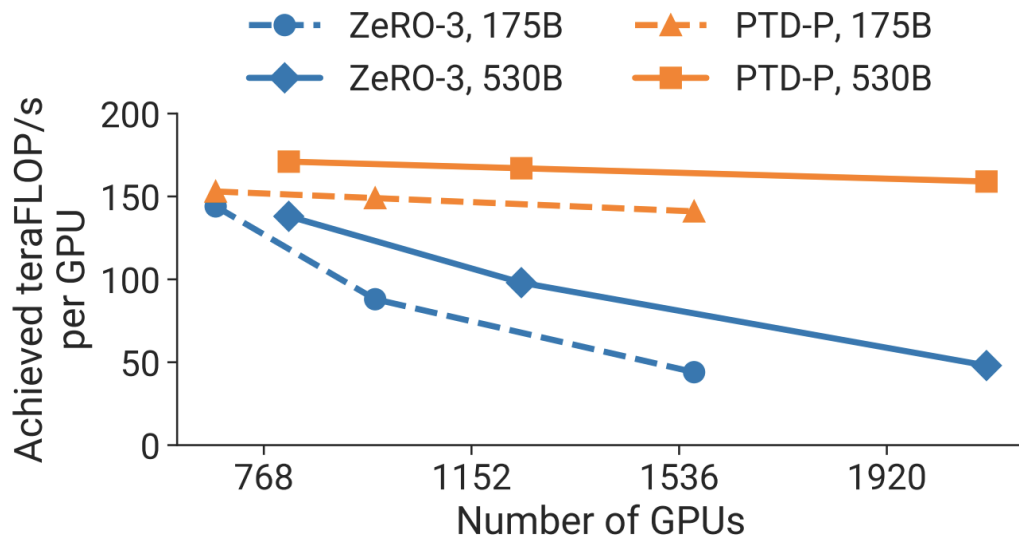
And there's diminishing returns to batch sizes



## Issues remain with data parallel – models don't fit

**Zero stages 1 and 2** don't let you scale memory

**Zero stage 3** is nice in principle, but *does not reduce activation memory*



Better ways to split up the model is needed...

# Beyond data parallel – model parallelism

**Scaling up in memory (without changing batch size) with model parallelism**

**What model parallelism is..**

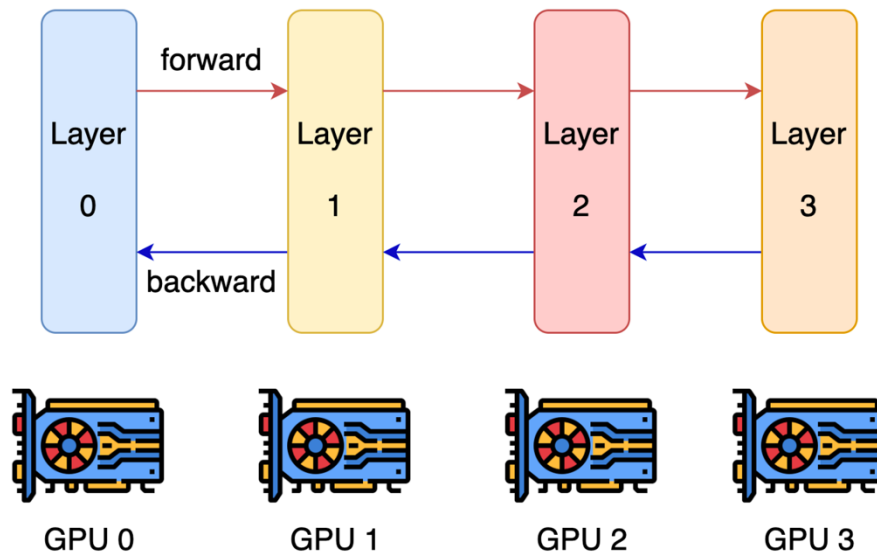
- It splits up the parameters across GPUs (like zero3)..
- But communicate activations (while zero3 sends params).

**We cover three different types of parallelism**

1. Pipeline parallel
2. Tensor parallel (+ Sequence parallel)
3. Expert parallel

These correspond to three different ways of cutting up the model.

## Layer-wise parallel

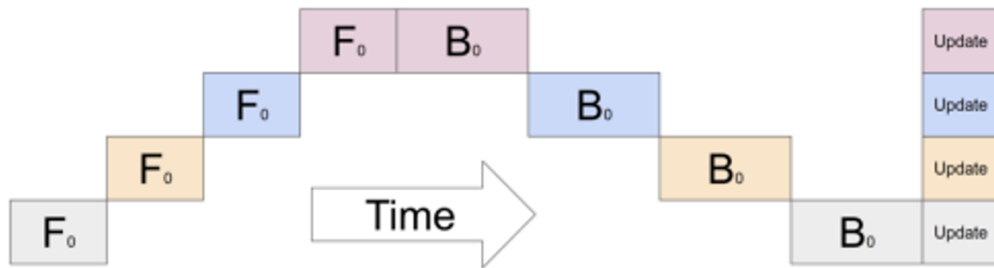


**Layer-wise parallel** cuts up layers, assigns some subset to GPUS.  
**Activations and partial gradients** are passed back and forth

# What's wrong with layer-wise parallel

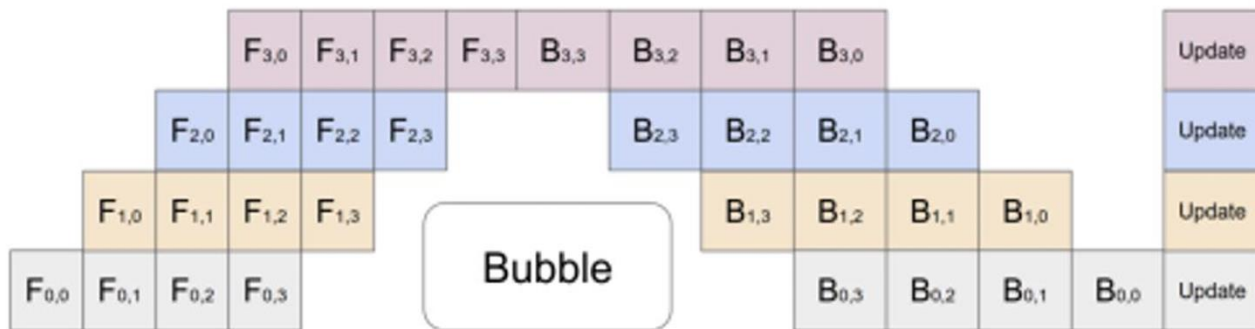
Utilization of layer-wise parallelism is *terrible*..

With  $n$  gpus, each gpu is active  $\frac{1}{n}$  of the time.



Each GPU is idling most of the time, waiting for the backward pass to propagate back

## A solution: pipeline parallel



**Solution: Pipeline-parallel.**

Process 'micro-batches' (in this case, 4).

Send off the first microbatch and start computing the second.

The ratio of bubble time to useful compute is ..  $\frac{n_{stages}-1}{n_{micro}}$  so we need a big batch size!

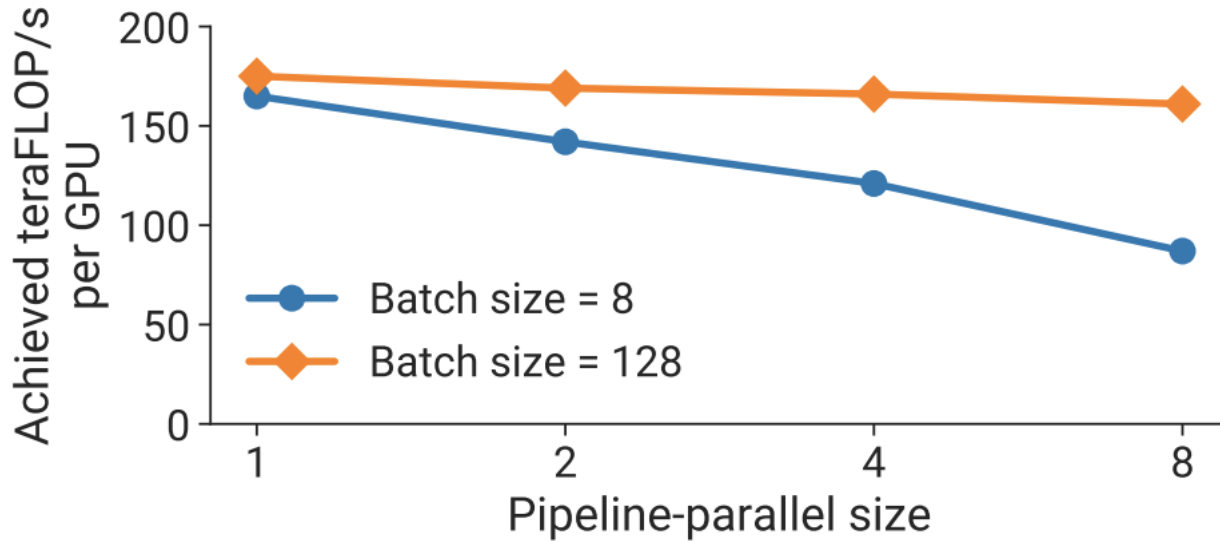
## Why pipeline parallel?

Pipelines seem terrible. Why do we do it?

1. Pipelines save memory (compared to DDP)
2. Pipelines can have good communication properties (compared to FDSP) – it depends only on activations ( $b \times s \times h$ ) and is *point to point*

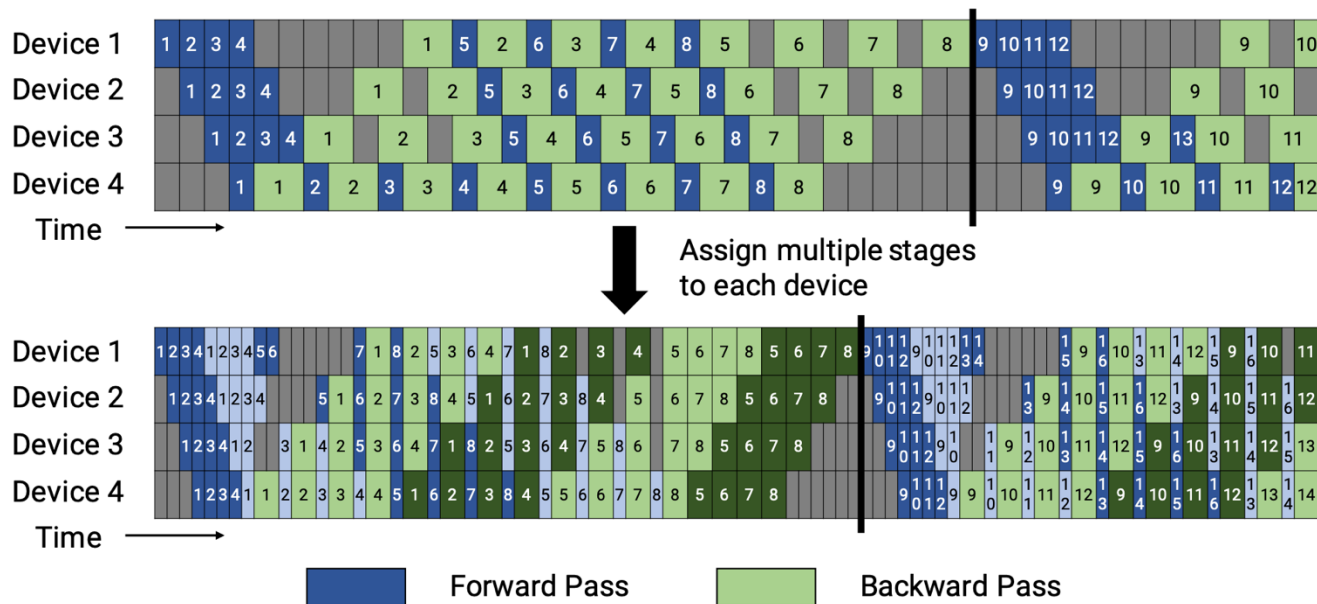
Generally, we will use pipelines on slower network links (i.e. inter-node) as a way to get better memory-wise scaling.

## Pipeline performance is highly dependent on batch size



Batch sizes are key to hiding the bubble – otherwise pipeline rapidly degrades perf

# Trading communication bandwidth for utilization



Some more crazy pipeline patterns can improve utilization, but at the cost of bandwidth

# 'Zero bubble' pipelining

## Split up backwards into two parts

1. Backpropagating activations (z,x)
2. Computing weight gradients (W)

The second part can be done whenever

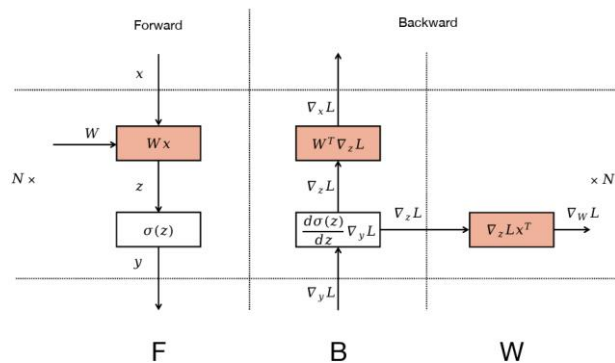


Figure 1: Computation Graph for MLP.



Figure 2: 1F1B pipeline schedule.

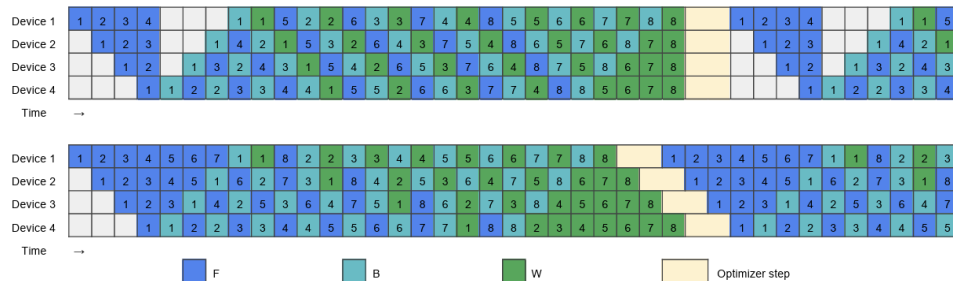
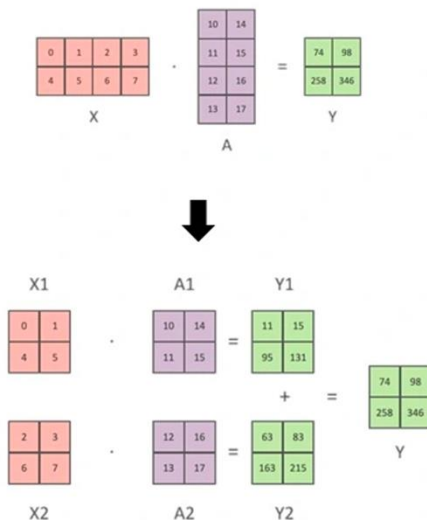


Figure 3: Handcrafted pipeline schedules, top: ZB-H1; bottom: ZB-H2

# Model parallel along the width axes

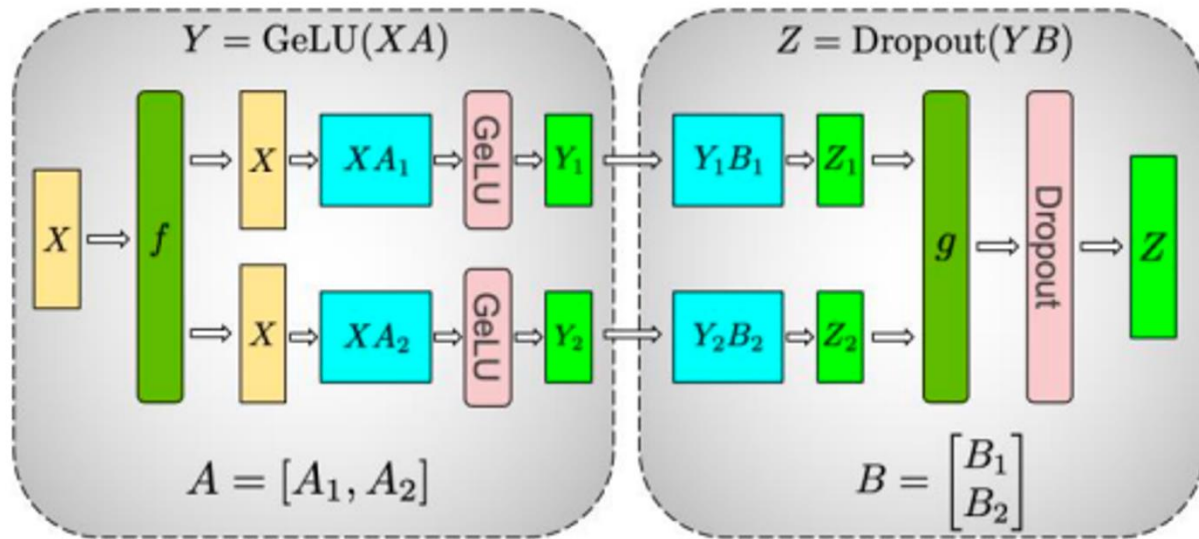
**Are there model parallel schemes with better utilization?**

We can think of pipeline parallel as cutting up along depth. What about width?



Simple matrix multiplication observation: decompose into submatrices, add partial sums

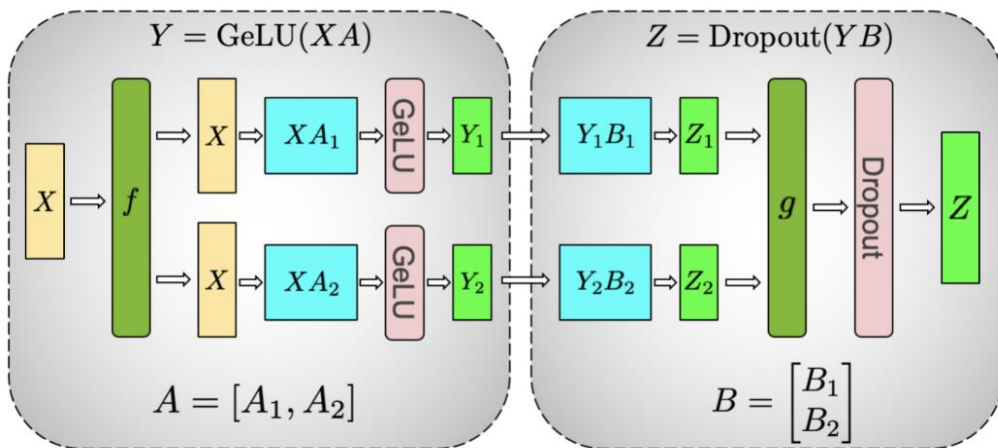
## Tensor parallel – GPUs have submatrices



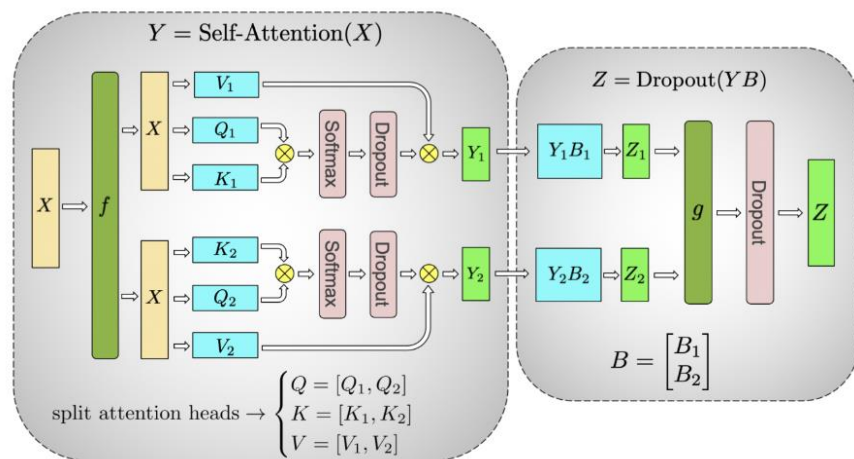
Assign columns ( $A_1, A_2$ ) and rows ( $B_1, B_2$ ) to separate GPUs.

- In the forward pass,  $f$  is the identity, and  $g$  is an all-reduce.
- In the backward pass,  $f$  is an all-reduce,  $g$  is the identity.

# Row vs Column tensor parallel



(a) MLP



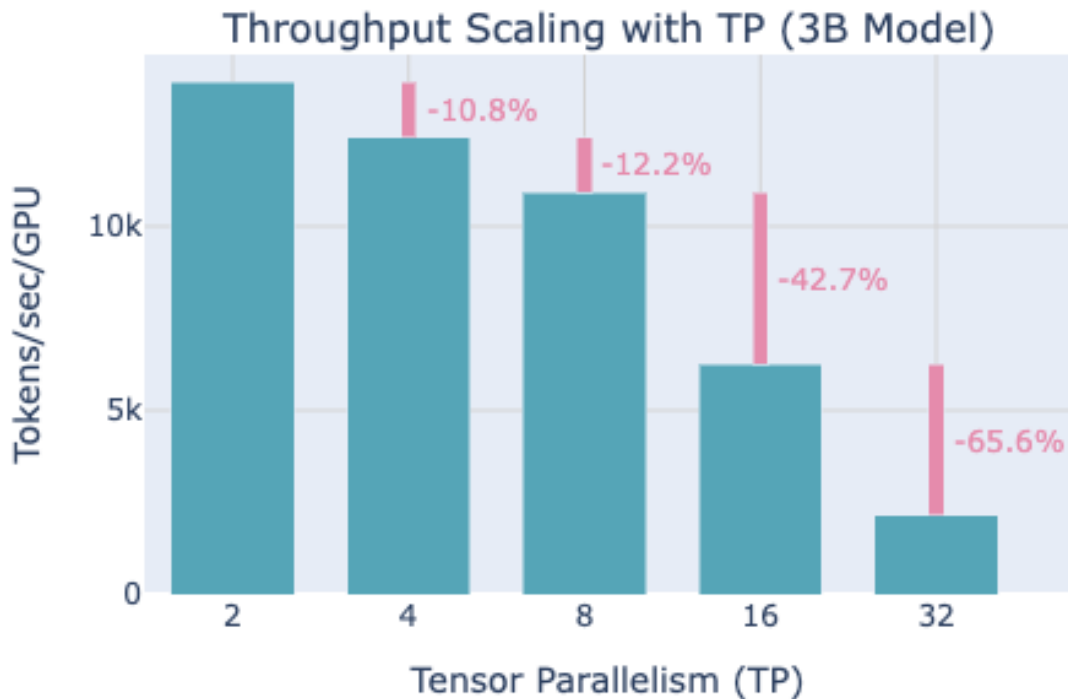
(b) Self-Attention

How do we split up in tensor parallel across a transformer block?

- Columnwise – QKV, up-projection
- Rowwise – Attention output, down-projection
- Replicated – norms, routers, etc

## When do we tensor parallel?

On GPUs, tensor parallel within a node (up to 8 GPUs) due to high speed interconnects.



# Tensor parallel – pros and cons vs pipeline parallel

How do things compare to pipeline parallel?

**Pros** – no bubble. If your network is fast enough, there's no waiting for others.

- low complexity – simple to 'wrap' models without major infra changes
- doesn't need large batch sizes to work well

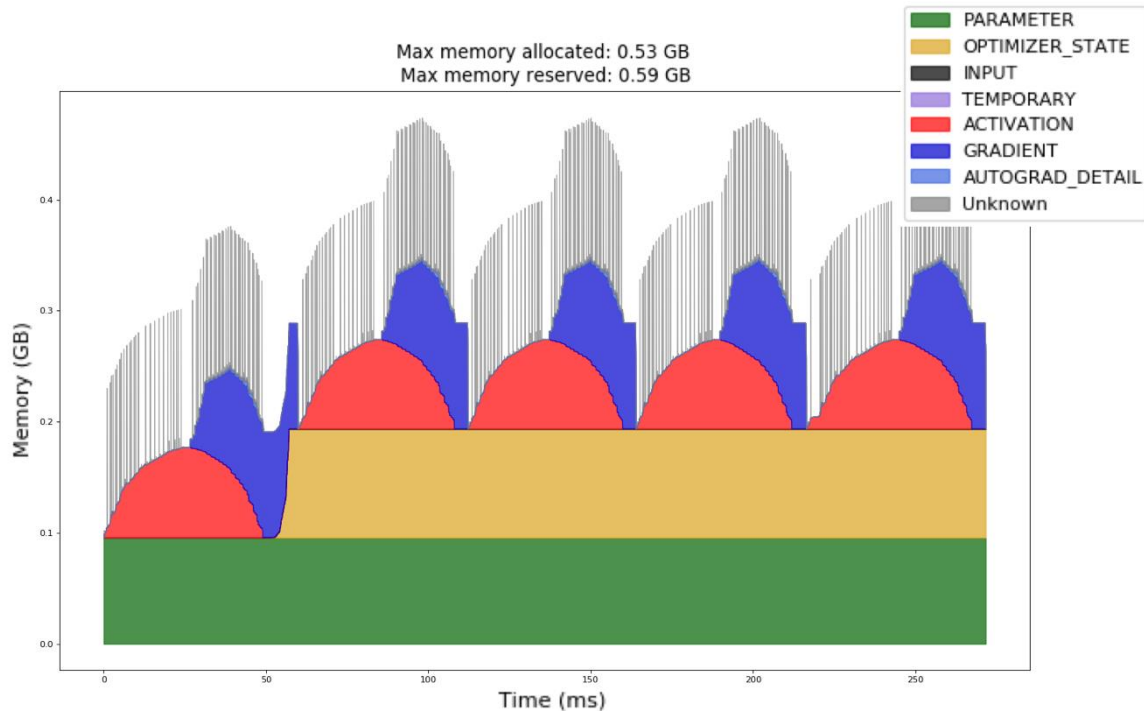
**Cons** – *much* larger communication than pipeline parallel.

- Pipeline:  $bsh$  point-to-point communication per microbatch
- Tensor:  $8bsh \left( \frac{n_{devices}-1}{n_{devices}} \right)$  per layer and *all-reduce* communication.

Use tensor parallel whenever we have low-latency, high-bandwidth interconnects

## A final complexity – memory is dynamic!

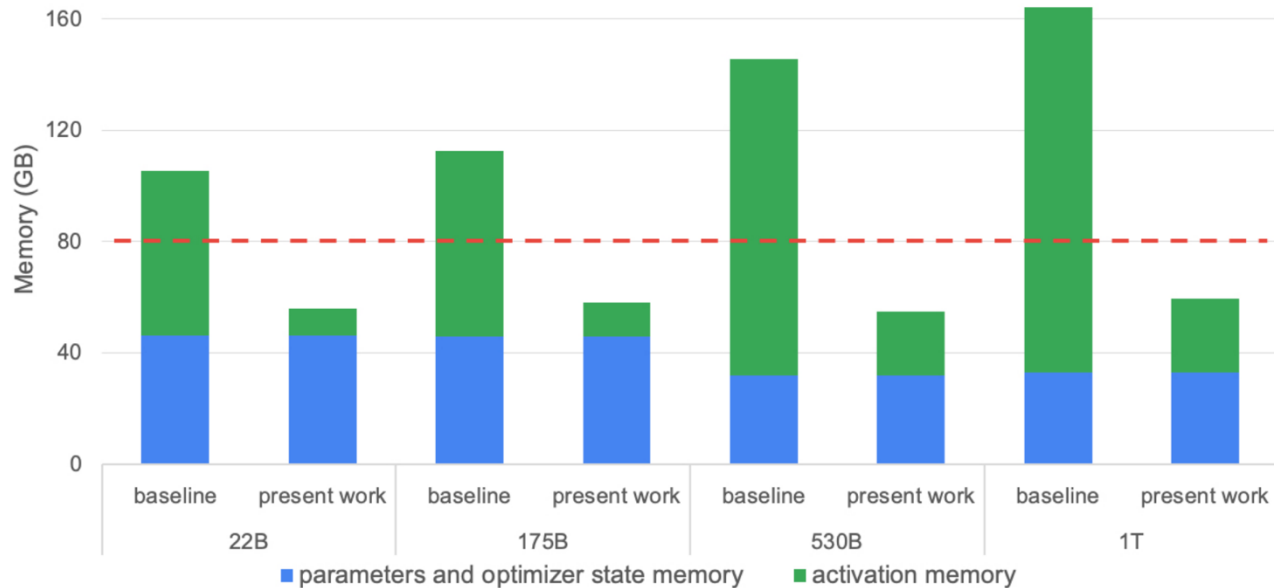
Memory isn't just the static bits, but also activations! This can be big



## A final complexity – activation memory

Thus far, we have only really discussed parameter memory.

Tensor and pipeline parallel can linearly reduce those..**but what about activations?**



# What's the activation memory per layer?

**Starting point:** activation memory needed if storing everything

$$\text{Activations memory per layer} = sbh \left( 34 + 5 \frac{as}{h} \right).$$

- The  $5 \frac{as}{h}$  terms come from the quadratic attention terms incl dropout
- As with flash attention, we can drop this term via recomputation

|     |                              |     |                        |
|-----|------------------------------|-----|------------------------|
| $a$ | number of attention heads    | $p$ | pipeline parallel size |
| $b$ | microbatch size              | $s$ | sequence length        |
| $h$ | hidden dimension size        | $t$ | tensor parallel size   |
| $L$ | number of transformer layers | $v$ | vocabulary size        |

## Activation under tensor parallel

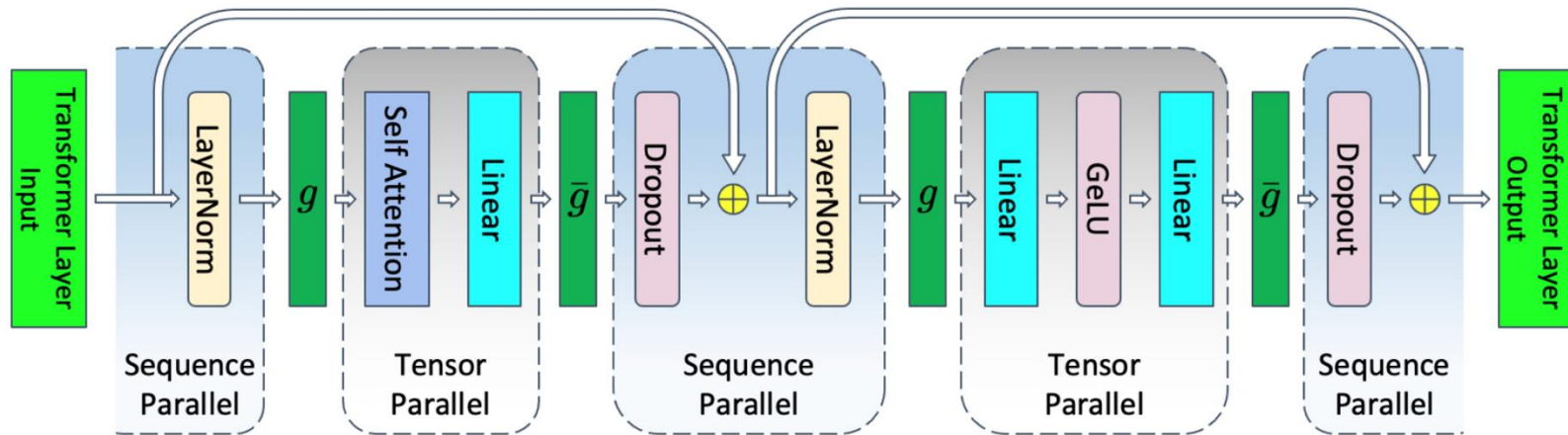
$$\underline{\text{Activations memory per layer}} = sbh \left( 10 + \frac{24}{t} + 5 \frac{as}{ht} \right)$$

**Tensor parallel** splits out the matrix multiplies in attention + MLP

The remaining **10** term is for the LayerNorm (4sbh), Dropout (2sbh), and inputs to the attention and MLP (4sbh). These terms alone will continue to grow with size

|     |                              |     |                        |
|-----|------------------------------|-----|------------------------|
| $a$ | number of attention heads    | $p$ | pipeline parallel size |
| $b$ | microbatch size              | $s$ | sequence length        |
| $h$ | hidden dimension size        | $t$ | tensor parallel size   |
| $L$ | number of transformer layers | $v$ | vocabulary size        |

## Making memory truly linear – sequence parallel



- Observation:** all the 10sbh terms are pointwise ops over the sequence  
... so split up the layer norm/dropout layers along the sequence axis.
- In the forward pass, ' $g$ ' is an all gather, ' $\bar{g}$ ' is reduce-scatter
  - In the backward pass, the two are reversed.

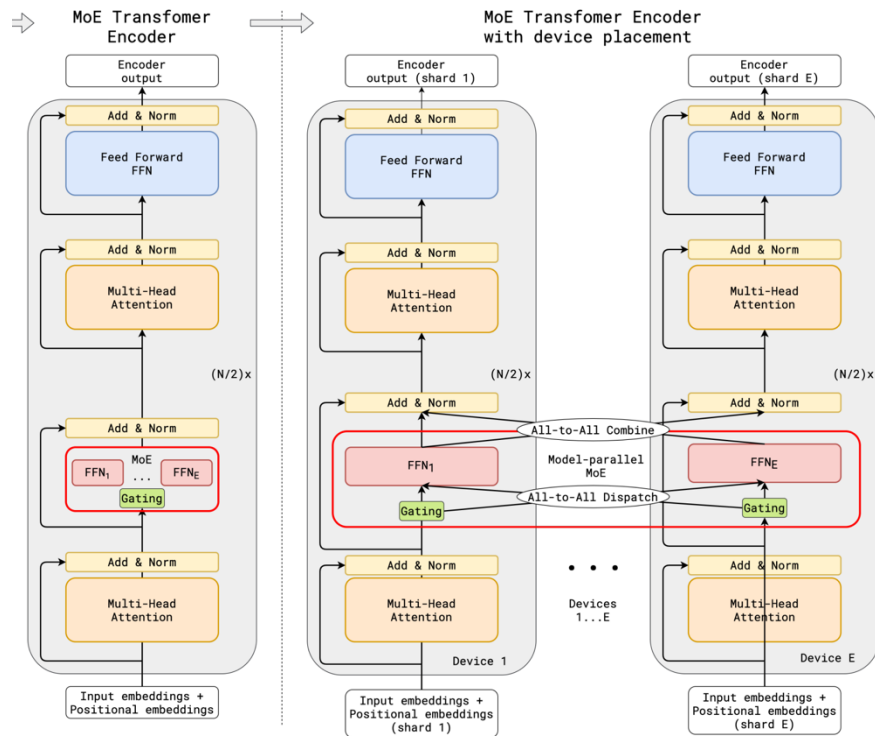
# Making activation memory fully scale with more machines

Putting it together to get full linear scaling for memory.

| Configuration                                                               | Activations Memory Per Transformer Layer                 |
|-----------------------------------------------------------------------------|----------------------------------------------------------|
| no parallelism                                                              | $sbh \left( 34 + 5 \frac{as}{h} \right)$                 |
| tensor parallel (baseline)                                                  | $sbh \left( 10 + \frac{24}{t} + 5 \frac{as}{ht} \right)$ |
| tensor + sequence parallel                                                  | $sbh \left( \frac{34}{t} + 5 \frac{as}{ht} \right)$      |
| tensor parallel +<br>selective activation recomputation                     | $sbh \left( 10 + \frac{24}{t} \right)$                   |
| tensor parallel + sequence parallel +<br>selective activation recomputation | $sbh \left( \frac{34}{t} \right)$                        |

# Expert parallelism

Instead of splitting up the matmul, split up the experts and route activations



## Why EP?

EP is *roughly* like TP in behavior for MLPs – high bandwidth, reduces activation

### Guideline 4: Prefer EP over TP for Expert Layers

| EP Advantages             | Details                                                                    |
|---------------------------|----------------------------------------------------------------------------|
| Better GEMM efficiency    | Larger local matrix sizes improve GPU utilization                          |
| Lower communication       | EP has less communication overhead than TP for MoE layers                  |
| Simpler computation graph | Easier to overlap communication with computation                           |
| Token permutation         | When <code>EP = num_experts</code> , local token permutation is eliminated |

**Example:** For Mixtral 8x7B, `EP8xTP1` outperforms `EP4xTP2`.

But, splitting matmuls can reduce efficiency vs routing activations

# Complexity – combining EP and others

Naively – we can compose EP and all other strategies..

**But there are important notes** – DP usually shares replicas with EP splits (so EP<DP)

DP and TP can interact badly to lower utilization

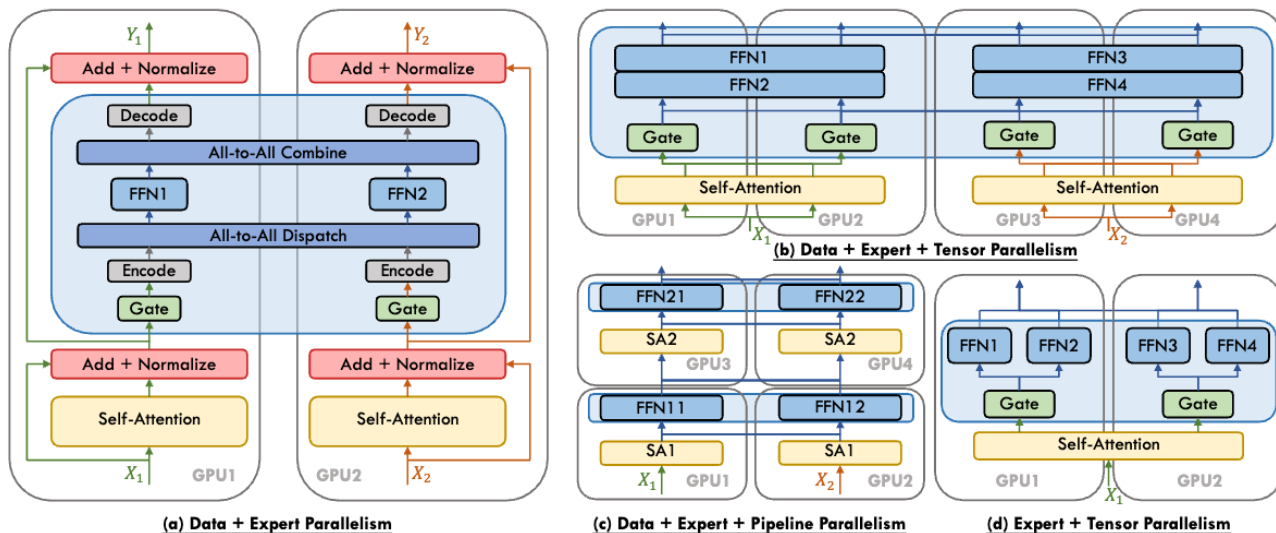


Fig. 8. Schematic depiction of diverse parallel strategies for MoE. For clarity and conciseness, this illustration omits some All-to-All, All-Reduce, Point-to-Point communication within parallelism, and Normalization, Encode, Decode, Gate in subfigures (b), (c), and (d).

# Complexity: Decoupling attention / expert parallelism

**Recall:** MoEs apply to the MLPs, not the attention (with some exotic exceptions)

This can lead to some imbalance between attention and MLPs for parallelism

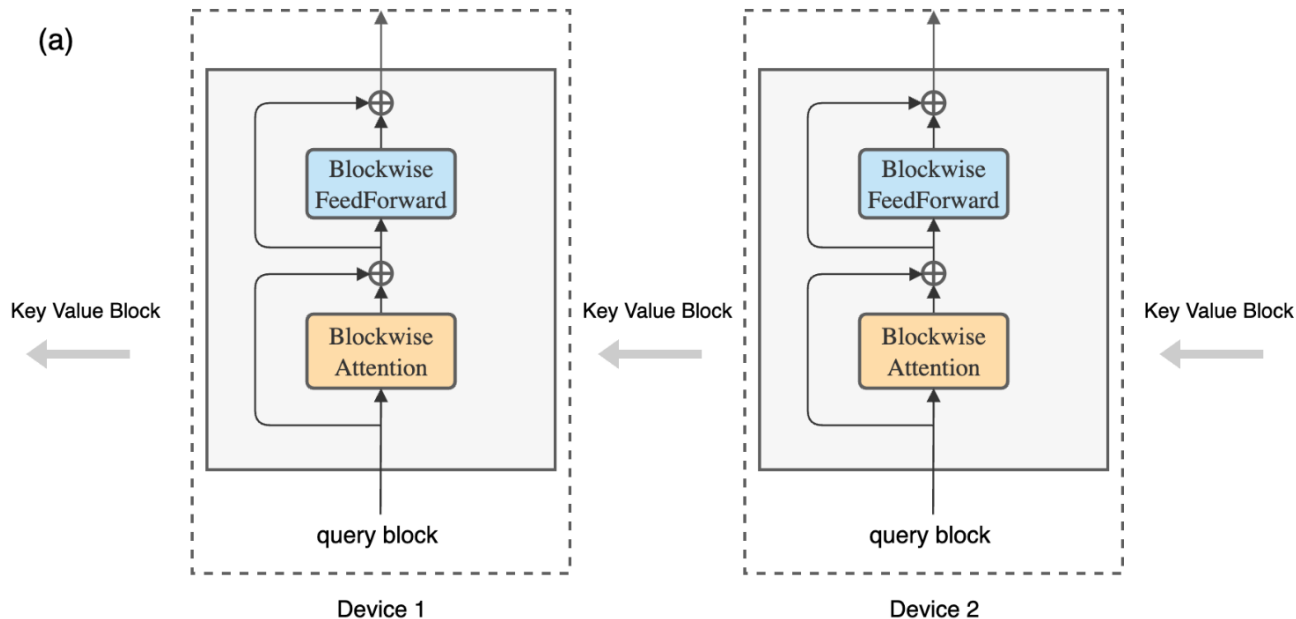
- High TP – useful for attention parallelization (since we can't use EP)
- Low TP – useful for MLP parallelization (we'd rather EP than TP)

What to do? More flexible parallelism strategies (i.e. in Megatron)

| MoE Parallel Folding is Megatron Core's solution that <b>decouples attention and MoE parallelism</b> : |                                    |                                      |
|--------------------------------------------------------------------------------------------------------|------------------------------------|--------------------------------------|
| Parallelism Group                                                                                      | Attention Layers                   | MoE Layers                           |
| Dimensions                                                                                             | $TP \times CP \times DP \times PP$ | $ETP \times EP \times EDP \times PP$ |

Separates out TP/CP/DP (for attention) and (ETP/EP/EDP) for MLPs

## Other parallelism strategies



**Context parallel / Ring attention** (split activations across GPUs in a long sequence)

# Recap: LLM parallelism table

| Method                      | Comm/Sync                                          | Param memory per rank                              | Activation / KV memory per rank                        | Main bandwidth cost                                                             | Scales global batch?                           | Easy to use?    |
|-----------------------------|----------------------------------------------------|----------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------|-----------------|
| DDP / ZeRO-1                | gradient all-reduce per step                       | <b>No param scaling</b><br>(ZeRO-1 only opt state) | <b>None</b>                                            | <b>Gradient traffic ~ <math>O(\text{params})</math></b>                         | <b>Yes, linear in DP</b>                       | <b>Very</b>     |
| FSDP / ZeRO-3               | Gradient all reduce – can be overlapped            | <b>~1/DP</b> for params/grads/optimizer state      | <b>None</b>                                            | <b>Param traffic ~ <math>O(\text{params})</math></b> , higher than DDP, overlap | <b>Yes, linear in DP</b>                       | <b>Moderate</b> |
| Pipeline parallel           | Activation between layers, <b>pipeline bubbles</b> | <b>~1/PP</b>                                       | Depends on pipeline buffers                            | <b>Activation traffic between stages</b>                                        | <b>No</b> , but needs microbatches             | <b>Hard</b>     |
| Tensor parallel             | <b>Blocking activation communication</b>           | <b>~1/TP</b> for TP-sharded weights                | <b>~1/TP</b> for relevant matmul activations w/ SP     | <b>Activation-sized collectives every block</b>                                 | <b>No</b>                                      | <b>Hard</b>     |
| Sequence / Context parallel | per-layer sequence-shard exchange                  | <b>None</b>                                        | <b>~1/SP or 1/CP</b> on sequence-side activations / KV | <b>Activation / KV communication</b>                                            | <b>No</b>                                      | <b>Hard</b>     |
| Expert parallel (MoE)       | <b>token dispatch - all-to-all per MoE</b>         | <b>~1/EP</b> for expert weights only               | <b>None</b>                                            | <b>Token-routing all-to-all</b>                                                 | <b>No</b> , but needs enough tokens per expert | <b>Hard</b>     |

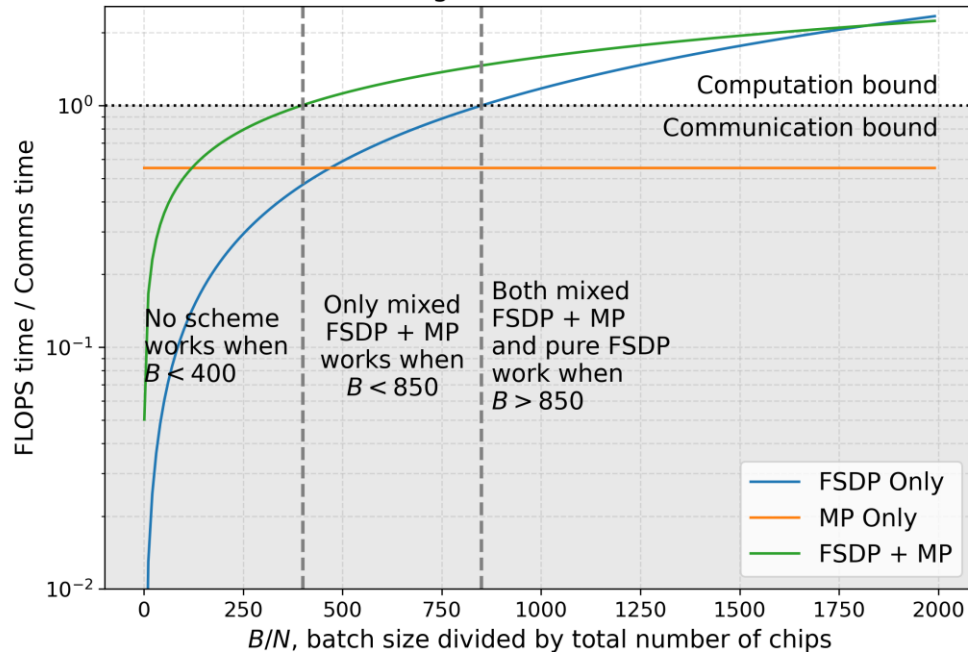
# Model vs Tensor parallel (TPU book)

## Key quantity

Global batch size (divided by GPU)

| Strategy  | Compute per layer<br>(ignoring gating einsum) | Comms per layer<br>(bytes, forward + backward pass) |
|-----------|-----------------------------------------------|-----------------------------------------------------|
| DP        | $4BDF/X + 8BDF/X$                             | $0 + 8DF$                                           |
| FSDP      | $4BDF/X + 8BDF/X$                             | $4DF + 8DF$                                         |
| MP        | $4BDF/Y + 8BDF/Y$                             | $4BD + 4BD$                                         |
| FSDP + MP | $4BDF/(XY) + 8BDF/(XY)$                       | $(4BD/X + 4DF/Y) + (8BD/X + 8DF/Y)$                 |

Batch-size scaling behavior of parallelization strategies on a 4x4x4 mesh

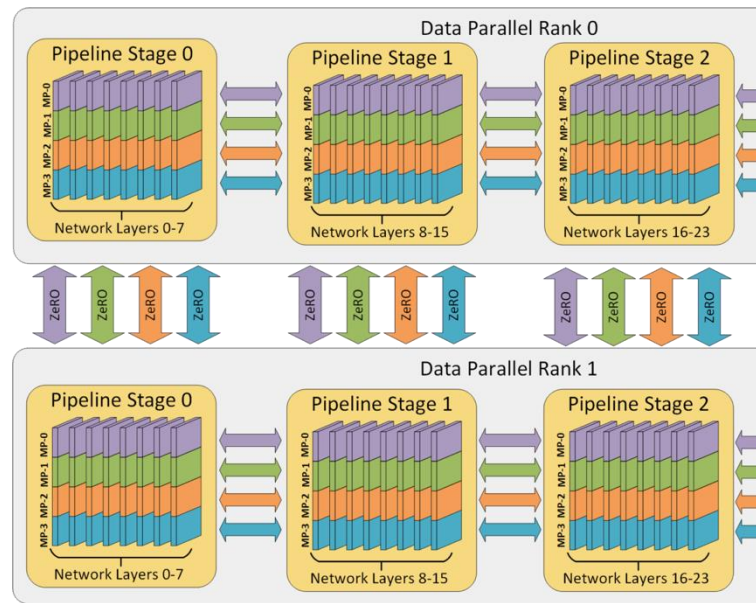


# '3D (4D) parallelism' – putting it all together

## Simple rules of thumb from the literature.

1. Until your model fits in memory..
  - Tensor / expert parallel up to GPUs / machine
  - Pipeline parallel across machines
  - › (Or use Zero-3, depending on BW)
2. Then until you run out of GPUs
  - Scale the rest of the way with data parallel

If your batch size is small.. gradient accumulate to trade higher batch sizes for better communication efficiency.



# Example from Megatron's current recommendations

## The “basics”

### Guideline 1: Minimize Model Parallelism, Maximize Data Parallelism

| Aspect | Recommendation                                                                                                                                      |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal   | Keep TP/EP/PP as small as possible while avoiding OOM                                                                                               |
| Why    | Model parallelism introduces communication overhead that hurts performance                                                                          |
| How    | Use distributed optimizer ( <code>--use-distributed-optimizer</code> ) to shard optimizer states across DP ranks, freeing memory for larger DP size |

### Guideline 2: Keep EP and TP Communication Within NVLink Domain

| Aspect  | Recommendation                                                                                             |
|---------|------------------------------------------------------------------------------------------------------------|
| Goal    | Ensure EP×TP fits within a single node (typically 8 GPUs)                                                  |
| Why     | EP and TP are communication-intensive; NVLink provides much higher bandwidth than cross-node interconnects |
| Scaling | When scaling beyond one node, prefer PP over expanding TP/EP across nodes                                  |

**Note:** For very large MoE models like DeepSeek-V3, the EP communication may exceed the NVLink bandwidth. In this case, consider using 1F1B A2A Overlap to overlap the EP communication.

### Guideline 3: Use Pipeline Parallelism (PP) for Multi-Node Scaling

| Aspect | Recommendation                                                                          |
|--------|-----------------------------------------------------------------------------------------|
| Goal   | Use PP to distribute layers across nodes while keeping EP×TP within NVLink              |
| VPP    | Enable Virtual Pipeline Parallelism to reduce pipeline bubbles when <code>PP ≥ 2</code> |
| Config | Set <code>--num-layers-per-virtual-pipeline-stage</code> to control VPP size            |

## Plus more advanced “extras”

### Guideline 4: Prefer EP over TP for Expert Layers

| EP Advantages                    | Details                                                                    |
|----------------------------------|----------------------------------------------------------------------------|
| <b>Better GEMM efficiency</b>    | Larger local matrix sizes improve GPU utilization                          |
| <b>Lower communication</b>       | EP has less communication overhead than TP for MoE layers                  |
| <b>Simpler computation graph</b> | Easier to overlap communication with computation                           |
| <b>Token permutation</b>         | When <code>EP = num_experts</code> , local token permutation is eliminated |

**Example:** For Mixtral 8x7B, `EP8×TP1` outperforms `EP4×TP2`.

### Guideline 5: Enable Context Parallelism (CP) for Long Sequences

| Aspect             | Recommendation                                                              |
|--------------------|-----------------------------------------------------------------------------|
| <b>When to use</b> | Sequence length $\geq$ 8K tokens                                            |
| <b>Key factor</b>  | CP efficiency depends on overlapping communication with computation         |
| <b>Config</b>      | Set <code>--context-parallel-size</code> to partition sequences across GPUs |

# Scaling strategies from Narayanan 2021

## Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM

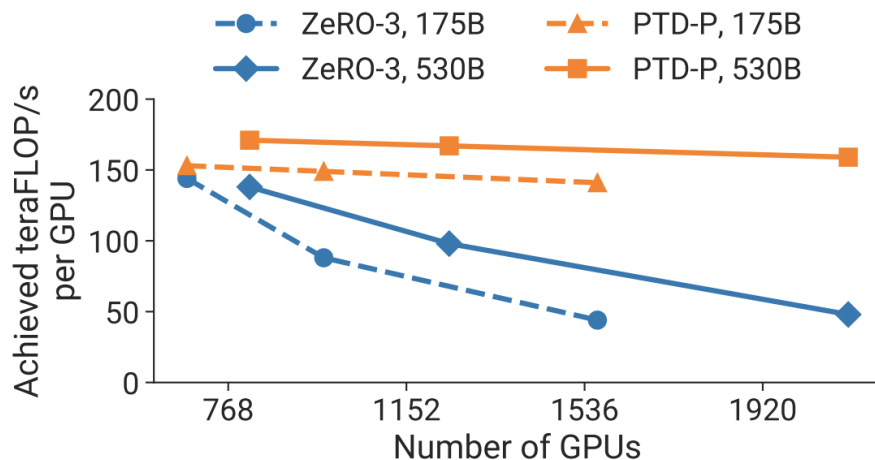
Deepak Narayanan<sup>1\*</sup>, Mohammad Shoeybi<sup>1</sup>, Jared Casper<sup>1</sup>, Patrick LeGresley<sup>1</sup>,  
Mostafa Patwary<sup>1</sup>, Vijay Korthikanti<sup>1</sup>, Dmitri Vainbrand<sup>1</sup>, Prethvi Kashinkunti<sup>1</sup>,  
Julie Bernauer<sup>2</sup>, Bryan Catanzaro<sup>1</sup>, Amar Phanishayee<sup>2</sup>, Matei Zaharia<sup>2</sup>  
<sup>1</sup>NVIDIA <sup>2</sup>Stanford University <sup>\*</sup>Microsoft Research

| Number of parameters (billion) | Attention heads | Hidden size | Number of layers | Tensor model-parallel size | Pipeline model-parallel size | Number of GPUs | Batch size | Achieved teraFLOP/s per GPU | Percentage of theoretical peak FLOP/s | Achieved aggregate petaFLOP/s | DP size |
|--------------------------------|-----------------|-------------|------------------|----------------------------|------------------------------|----------------|------------|-----------------------------|---------------------------------------|-------------------------------|---------|
| 1.7                            | 24              | 2304        | 24               | 1                          | 1                            | 32             | 512        | 137                         | 44%                                   | 4.4                           | 32      |
| 3.6                            | 32              | 3072        | 30               | 2                          | 1                            | 64             | 512        | 138                         | 44%                                   | 8.8                           | 32      |
| 7.5                            | 32              | 4096        | 36               | 4                          | 1                            | 128            | 512        | 142                         | 46%                                   | 18.2                          | 32      |
| 18.4                           | 48              | 6144        | 40               | 8                          | 1                            | 256            | 1024       | 135                         | 43%                                   | 34.6                          | 32      |
| 39.1                           | 64              | 8192        | 48               | 8                          | 2                            | 512            | 1536       | 138                         | 44%                                   | 70.8                          | 32      |
| 76.1                           | 80              | 10240       | 60               | 8                          | 4                            | 1024           | 1792       | 140                         | 45%                                   | 143.8                         | 32      |
| 145.6                          | 96              | 12288       | 80               | 8                          | 8                            | 1536           | 2304       | 148                         | 47%                                   | 227.1                         | 24      |
| 310.1                          | 128             | 16384       | 96               | 8                          | 16                           | 1920           | 2160       | 155                         | 50%                                   | 297.4                         | 15      |
| 529.6                          | 128             | 20480       | 105              | 8                          | 35                           | 2520           | 2520       | 163                         | 52%                                   | 410.2                         | 9       |
| 1008.0                         | 160             | 25600       | 128              | 8                          | 64                           | 3072           | 3072       | 163                         | 52%                                   | 502.0                         | 6       |

### Notes

- Tensor parallel first up to 8, then caps out at 8.
- Pipeline parallel goes up to make the model fit.
- Data parallel gradually decreases with scale, with the largest model having DP=6

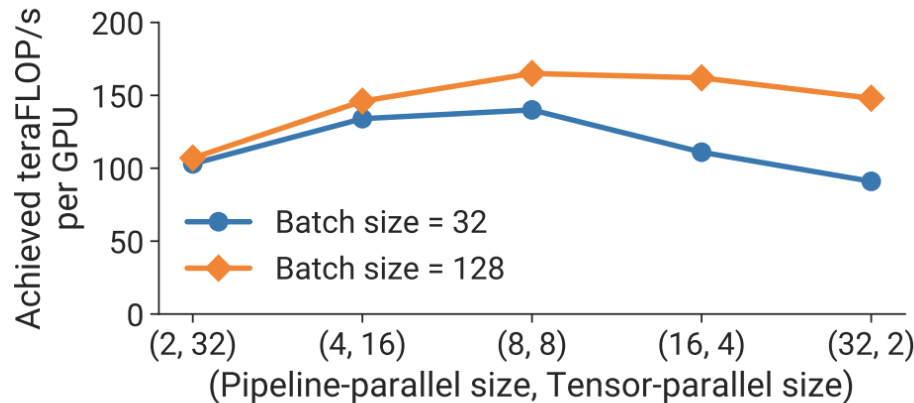
## Careful '3D' parallelism gives linear gains



**Figure 10: Throughput per GPU of PTD-P and ZeRO-3 for two different GPT models (the 175B GPT-3 model is shown with dotted lines, and the 530B model is shown with solid lines). Global batch sizes are fixed and ZeRO-3 is used without any model parallelism.**

More GPUS, same, flat utilization!

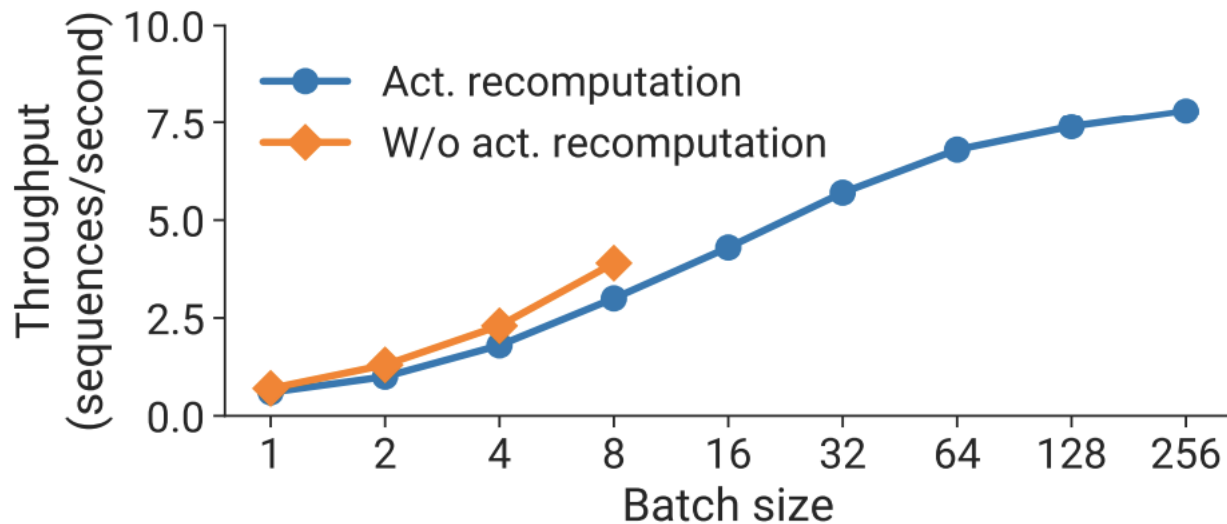
## Tensor parallel = 8 is often optimal



**Figure 13: Throughput per GPU of various parallel configurations that combine pipeline and tensor model parallelism using a GPT model with 162.2 billion parameters and 64 A100 GPUs.**

When parallelizing across 64 machines – it's best to use a 8 x 8 configuration.

## Activation recomputation can pay for itself (via memory)



Activation recomputation enables larger batches, improving throughput (t=8, p=16)

# Recent LMs – what do they do?

## 3.1 Distributed Training Framework

We train our models using the *ZeRO* optimizer strategy (Rajbhandari et al., 2019) via PyTorch's FSDP framework (Zhao et al., 2023), which reduces memory consumption by sharding the model weights and their corresponding optimizer state across GPUs. At the 7B scale, this enables training with a micro-batch size of 4096 tokens per GPU on our hardware (see Section 3.4). For OLMo-1B and -7B models, we use a constant global batch size of approximately 4M tokens (2048 instances, each with a sequence length of 2048 tokens). For OLMo-65B model (currently training), we use a batch size warmup that starts at approximately 2M tokens (1024 instances), then doubles every 100B tokens until reaching approximately 16M tokens (8192 instances).

To improve throughput, we employ mixed-precision training (Micikevicius et al., 2017) through FSDP's built-in settings and PyTorch's amp module. The latter ensures that certain operations like the softmax always run in full precision to improve stability, while all other operations run in half-precision with the bfloat16 format. Under our specific settings, the sharded model weights and optimizer state local to each GPU are kept in full precision. The weights within each transformer block are only cast to bfloat16 when the full-sized parameters are materialized on each GPU during the forward and backward passes. Gradients are reduced across GPUs in full precision.

**Dolma** – 7B model, FDSP (probably fits intra-node)

# DeepSeek

## 2.4. Infrastructures

We use an efficient and light-weight training framework named HAI-LLM (High-flyer, 2023) to train and evaluate large language models. Data parallelism, tensor parallelism, sequence parallelism, and 1F1B pipeline parallelism are integrated into this framework as done in Megatron (Korthikanti et al., 2023; Narayanan et al., 2021; Shoeybi et al., 2019). We also leverage the flash attention (Dao, 2023; Dao et al., 2022) technique to improve hardware utilization. ZeRO-1 (Rajbhandari et al., 2020) is exploited to partition optimizer states over data parallel ranks. Efforts are also made to overlap computation and communication to minimize additional waiting overhead, including the backward procedure of the last micro-batch and reduce-scatter operation in ZeRO-1, and GEMM computation and all-gather/reduce-scatter in sequence parallel. Some layers/operators are fused to speed up training, including LayerNorm, GEMM whenever possible, and Adam updates. To improve model training stability, we train the model in bf16 precision but accumulate gradients in fp32 precision. In-place cross-entropy is performed to reduce GPU memory consumption, i.e.: we convert bf16 logits to fp32 precision on the fly in the cross-entropy CUDA kernel (instead of converting it beforehand in HBM), calculate the corresponding bf16 gradient, and overwrite logits with its gradient.

**DeepSeek** – ZeRO stage 1 with Tensor, Sequence, and Pipeline parallel

**V3** – PP (16), EP (64-way, 8 nodes), ZeRO stage 1

EP uses 1F1B A2A Overlap

**Performance and Cost Efficiency**

*Memory and communication* restrictions are the two major technical challenges of large scale model training requiring integrated solutions beyond adding more GPUs. We use and improve upon the following techniques to tackle the memory and communication restrictions: (1) ZeRO-1 [60] to remove the memory consumption by partitioning optimizer states cross data-parallel processes; (2) tensor parallel combined with pipeline parallel [70] within each compute node to avoid inter-node communication bottleneck, and the 3D parallel strategy is well designed and optimized to avoid using activation checkpointing and minimize the pipeline bubbles; (3) kernel fusion techniques like flash attention [15] [14] and JIT kernels to reduce redundant global memory access and consumption; (4) topology-aware resource allocation (ranking strategy) to minimize the communication across different layers of switches, which is the limitation of a typical fat-tree-topology.

**Yi** - ZeRO stage 1 + Tensor + Pipeline parallel

Yi-lightning (2025) – Tensor replaced by Expert parallelism

# Llama3 405B

| GPUs   | TP | CP | PP | DP  | Seq. Len. | Batch size/DP | Tokens/Batch | TFLOPs/GPU | BF16 MFU |
|--------|----|----|----|-----|-----------|---------------|--------------|------------|----------|
| 8,192  | 8  | 1  | 16 | 64  | 8,192     | 32            | 16M          | 430        | 43%      |
| 16,384 | 8  | 1  | 16 | 128 | 8,192     | 16            | 16M          | 400        | 41%      |
| 16,384 | 8  | 16 | 16 | 8   | 131,072   | 16            | 16M          | 380        | 38%      |

(Stage 1, small bsz training, Stage 2 pretraining, Stage 3 long-context)

**Network-aware parallelism configuration.** The order of parallelism dimensions, [TP, CP, PP, DP], is optimized for network communication. The innermost parallelism requires the highest network bandwidth and lowest latency, and hence is usually constrained to within the same server. The outermost parallelism may spread across a multi-hop network and should tolerate higher network latency. Therefore, based on the requirements for network bandwidth and latency, we place parallelism dimensions in the order of [TP, CP, PP, DP]. DP (*i.e.*, FSDP) is the outermost parallelism because it can tolerate longer network latency by asynchronously prefetching sharded model weights and reducing gradients. Identifying the optimal parallelism configuration with minimal communication overhead while avoiding GPU memory overflow is challenging. We develop a memory consumption estimator and a performance-projection tool which helped us explore various parallelism configurations and project overall training performance and identify memory gaps effectively.

# Llama 3 405B

**Side note** – Lots of GPU failures at this scale!

| Component                      | Category              | Interruption Count | % of Interruptions |
|--------------------------------|-----------------------|--------------------|--------------------|
| Faulty GPU                     | GPU                   | 148                | 30.1%              |
| GPU HBM3 Memory                | GPU                   | 72                 | 17.2%              |
| Software Bug                   | Dependency            | 54                 | 12.9%              |
| Network Switch/Cable           | Network               | 35                 | 8.4%               |
| Host Maintenance               | Unplanned Maintenance | 32                 | 7.6%               |
| GPU SRAM Memory                | GPU                   | 19                 | 4.5%               |
| GPU System Processor           | GPU                   | 17                 | 4.1%               |
| NIC                            | Host                  | 7                  | 1.7%               |
| NCCL Watchdog Timeouts         | Unknown               | 7                  | 1.7%               |
| Silent Data Corruption         | GPU                   | 6                  | 1.4%               |
| GPU Thermal Interface + Sensor | GPU                   | 6                  | 1.4%               |
| SSD                            | Host                  | 3                  | 0.7%               |
| Power Supply                   | Host                  | 3                  | 0.7%               |
| Server Chassis                 | Host                  | 2                  | 0.5%               |
| IO Expansion Board             | Host                  | 2                  | 0.5%               |
| Dependency                     | Dependency            | 2                  | 0.5%               |
| CPU                            | Host                  | 2                  | 0.5%               |
| System Memory                  | Host                  | 2                  | 0.5%               |

**Table 5 Root-cause categorization of unexpected interruptions during a 54-day period of Llama 3 405B pre-training.** About 78% of unexpected interruptions were attributed to confirmed or suspected hardware issues.

# Gemma 2

**For 2, 9, 27B models**

ZeRO-3, MP (=TP+SP), DP

## 3.3. Compute Infrastructure

We train our models with TPUv4, TPUv5e, and TPUv5p as outlined in [Table 3](#). For the 2B model, we train on a 2x16x16 configuration of TPUv5e, totaling 512 chips, with 512-way data replication and 1-way model sharding. For the 9B model, we train on an 8x16x32 configuration of TPUv4, totaling 4096 chips, with 1024-way data replication and 4-way model sharding. For the 27B model, we train on an 8x24x32 configuration of TPUv5p, totaling 6144 chips, with 768-way data replication and 8-way model sharding.

The optimizer state is further sharded using techniques similar to ZeRO-3 ([Ren et al., 2021](#)). For scales beyond a single pod, we perform a data-replica reduction over the data center network, using the Pathways approach of [Barham et al. \(2022\)](#). We also use the 'single controller' programming paradigm of Jax ([Roberts et al., 2023](#)) and Pathways ([Barham et al., 2022](#)). As in Gemma 1, we use the GSPMD partitioner ([Xu et al., 2021](#)) for training step computation and the MegaScale XLA compiler ([XLA, 2019](#)).

# Mixtral 8x22B

From megatron docs –

TP / PP / CP / EP (4/4/1/8) – DP likely 2 (to add up to 256GPUs)

| Mixture-of-Experts Models |       |      |    |    |    |    |                              |
|---------------------------|-------|------|----|----|----|----|------------------------------|
| Model                     | Size  | GPUs | TP | PP | CP | EP | Configuration Notes          |
| Mixtral                   | 8x7B  | 64   | 1  | 4  | 1  | 8  | EP=8 for 8 experts           |
| Mixtral                   | 8x22B | 256  | 4  | 4  | 1  | 8  | TP+PP+EP for large MoE       |
| DeepSeek-V3               | 671B  | 1024 | 2  | 16 | 1  | 64 | Massive MoE with 256 experts |

# Nemotron 3 Super 120B-A12B

## From the long-context extension section

Similar to Nemotron 3 Nano, we added a long-context phase (LC-Phase) at the end of pretraining. In the LC-Phase, we performed continuous pretraining (CPT) to equip the base model with long-context ability. We used a constant learning rate of  $4.5 * 10^{-6}$  and global batch size of 16. We used 64-way context parallelism, 2-way tensor parallelism, and 64-way expert parallelism to train on GB200 GPUs. We reused the long-context document QA dataset from Nemotron 2 & 3 Nano. We allocated the document QA data to 20% in the Phase LC data blend, with the remaining 80% being downsampled Phase 2 data. We initially performed CPT on 1,048,576 (1m) context length. Such stage lasted for 34 billion tokens. Following that we added another stage to alternately train on both 1m and 4k sequences in order to mitigate the minor impact we observed on the math-related benchmarks. The second stage lasted for 17 billion tokens.

TP / PP / CP / EP (2/0/64/64)

# Qwen 3

225B-A22B and 30B-A3B

## Parallelism Configuration

| Model           | Mode      | TP | PP | EP | Total GPUs | Use Case                              |
|-----------------|-----------|----|----|----|------------|---------------------------------------|
| Qwen3-30B-A3B   | Pretrain  | 1  | 1  | 8  | 8          | Pre-training (single node)            |
| Qwen3-30B-A3B   | Full SFT  | 1  | 1  | 8  | 8          | Full supervised finetuning            |
| Qwen3-30B-A3B   | LoRA/DoRA | 1  | 1  | 8  | 8          | PEFT finetuning (single node)         |
| Qwen3-235B-A22B | Pretrain  | 2  | 8  | 32 | 512        | Pre-training (64 nodes)               |
| Qwen3-235B-A22B | Full SFT  | 2  | 8  | 32 | 512        | Full supervised finetuning (64 nodes) |
| Qwen3-235B-A22B | LoRA/DoRA | 2  | 8  | 32 | 512        | PEFT finetuning (64 nodes)            |

| Workload family   | Hardware | Typical band                                   | Representative shape               |
|-------------------|----------|------------------------------------------------|------------------------------------|
| DSV3, large-scale | H100     | low-to-mid hundreds TFLOPS/GPU, high-teens MFU | TP2, EP64, PP8, DeepEP             |
| DSV3, large-scale | B200     | high-hundreds TFLOPS/GPU, mid-teens MFU        | TP1, EP32, PP8, DeepEP             |
| DSV3, large-scale | GB200    | around 1K TFLOPS/GPU, low-20s MFU              | TP1, EP64, PP4, HybridEP           |
| DSV3, large-scale | GB300    | above the GB200 band, often mid-20s MFU        | TP1, EP64, PP4, HybridEP           |
| Qwen3 235B        | H100     | low-300s TFLOPS/GPU, around 30% MFU            | TP2, EP32, PP8, DeepEP             |
| Qwen3 235B        | GB200    | high-hundreds TFLOPS/GPU in tuned runs         | TP1 or TP2, EP32-64, PP4, HybridEP |
| Qwen3 30B         | H100     | low-200s TFLOPS/GPU                            | TP1, EP8, PP1, DeepEP              |
| Qwen3-Next 80B    | GB200    | low-300s TFLOPS/GPU in BF16-class runs         | TP1, EP32, PP2, HybridEP           |

Primarily expert parallel up to 8 GPUs, 2/8/32 parallelism for larger models

## Overview table of model parallelism

|                                      | DP         | TP/SP | EP | PP | CP |
|--------------------------------------|------------|-------|----|----|----|
| Deepseek                             | ?? (Zero1) | 1     | 8  | 16 | ?? |
| Deepseek v3                          | ?? (Zero1) | 1     | 64 | 16 | ?? |
| Yi                                   | ?? (Zero1) | >0    | 1  | >0 | ?? |
| Llama3 405B                          | 128        | 8     | 0  | 16 | 1  |
| Gemma 2                              | 768        | 8     | 0  | 0  | 0  |
| Mixtral 8x22<br>(megatron)           | 2          | 4     | 8  | 4  | 1  |
| Nemotron 3<br>120B (long<br>context) | ??         | 2     | 64 | ?? | 64 |
| Qwen 3<br>(megatron)                 | ??         | 2     | 32 | 8  | 1  |

**Patterns** – TP generally  $\leq 8$ . EP can be big (but hard!). Long context phases use large CP

## Recap for the whole lecture

- ❖ Scaling beyond a certain point requires multi-gpu, multi-node parallelism
- ❖ No single solution to the parallelism problem (probably want all 3 approaches)
- ❖ Simple, interpretable rules of thumb for combining different forms of parallelism